

BranchCARET Model Checking of Self Modifying Code

Abstract—Binary code analysis is an important task, especially for malware detection. Model checking is one of the static analysis techniques that can reason about behaviour of a program. However, when working with binary code, it is crucial to take self-modifying instructions into account. We consider Self-Modifying Pushdown Systems (SM-PDS) as an abstraction for self-modifying binary code. To specify malicious behaviour, LTL, CTL, and CARET have been used previously. However, sometimes, to specify malicious behaviour, one needs both branching features of CTL and call-return relations of CARET. BranchCARET was proposed to combine these features, and several BranchCARET model checking algorithms for standard Pushdown Systems were proposed. In this work, we present an efficient algorithm for BranchCARET model checking of *self-modifying* PDS. This task is not trivial, because BranchCARET allows to reason about past states, while self-modification can change the set of applicable transitions, such that the transitions are not the same in the past and in the present. We tackle this by recording self-modifications on the stack. We have implemented our algorithm in a tool and obtained promising results. In particular, we were able to detect several self-modifying malwares.

Index Terms—Branching CARET model checking, pushdown systems, self modifying code, malware detection.

I. INTRODUCTION

Analysis of binary programs is an important issue when one does not have an access to a program’s original source code. This is often the case in malware analysis and analysis of proprietary software. However, binary programs are often obfuscated using self-modification. Therefore, it is important to analyse *binary* programs with *self-modification*.

In this work, we consider verification of self-modifying binary code. First, let us demonstrate an example of self-modifying code in Listing 1. This code is supposed to run on an x86 architecture under Windows operating system.

The binary program starts execution at the address 0x14 that contains the bytecode eb00. This bytecode corresponds to the assembly instruction `jmp 0x16`, which jumps to the address 0x16. The address 0x16 contains binary code c6051500000009, which corresponds to the assembly instruction `mov [0x15], 0x09`. This instruction replaces the byte stored at address 0x15 with the byte 0x09. Note that the address 0x15 corresponds to the byte 00 inside of the bytecode eb00 stored at the address 0x14. In other words, the instruction `mov [0x15], 0x09` replaces the bytecode eb00 with the bytecode eb09. The new code eb09 corresponds to the assembly instruction `jmp 0x1f`, which jumps to the address 0x1f. Thus, the `mov` instruction is a self-modifying instruction that replaces the instruction `jmp 0x16`

#	Address	Bytecode	Assembly
	0x14	eb00	<code>jmp 0x16</code>
	0x16	c6051500000009	<code>mov [0x15], 0x09</code>
	0x1d	ebf5	<code>jmp 0x14</code>
	0x1f	ff1500000000	<code>call [InternetOpenA]</code>

Listing 1. Example of self-modifying code.

#	Address	Assembly	
	0x14	<code>jmp 0x16</code>	<i>#-- executing -> jmp 0x1f</i>
			<i># mov [0x15], 0x09</i>
	0x16	<code>mov [0x15], 0x09</code>	
	0x1d	<code>jmp 0x14</code>	
	0x1f	<code>call [InternetOpenA]</code>	

Listing 2. Example from Listing 1 after self-modification.

with the instruction `jmp 0x1f`. After the `mov` instruction, the program executes the address 0x1d, which contains the bytecode ebf5, corresponding to the assembly instruction `jmp 0x14`, which jumps to the address 0x14. Now, the address 0x14 contains the freshly modified instruction `jmp 0x1f` instead of the original `jmp 0x16`. The binary code after self-modification is presented in Listing 2. The new instruction jumps to the address 0x1f, which contains the bytecode ff1500000000. This bytecode corresponds to the instruction `call [InternetOpenA]`, which calls the Windows API function *InternetOpenA*. This function accesses the internet and opens the provided URL. However, if one analyses this binary without considering self-modification, the instruction calling the *InternetOpenA* function would not appear to be reachable. The program would appear to consist of only the infinite loop at addresses 0x14, 0x16, and 0x1d. Therefore, it is important to consider self modification when analysing binary code.

To verify self-modifying binaries, we consider model checking, which allows to reason about behaviour of the program without executing it. Previous studies that consider model checking [17], [23] have successfully proved that binary code can be efficiently model checked for the purpose of malware detection. However, one requires an adequate abstraction for model checking. Pushdown System (PDS) is a natural abstraction for binary programs [23]. To model self-modifying behaviour, we consider Self-Modifying Pushdown Systems (SM-PDS) proposed in [24], which also proposes reachability analysis, LTL, and CTL model checking of SM-PDS. However, LTL and CTL is sometimes not enough for malware detection. It is important to reason about calls and their corresponding returns to have understanding of return

values and parameters of functions. Temporal Logic for Calls and Returns (CARET) [2] was introduced for this reason. Later, [20] proposed CARET model checking of PDS. However, CARET can reason only about one possible run of the program. And there are malicious behaviours that cannot be described with CARET [21]. Instead, one requires to talk about which branches a program may take. Such expressivity is obtainable only with branching time logics. To solve this, [21] and [16] proposed two branching versions of CARET (BranchCARET) simultaneously. Both of these works propose a new branching time logic that can reason about function calls and returns. However, the authors considered only model checking on standard, non self-modifying Pushdown Systems, which may not be enough in the case of self-modifying code.

We go one step further and present a *direct* and *efficient* algorithm for BranchCARET model checking of SM-PDS, which is a natural abstraction for self-modifying binary code. To perform model checking and compute whether the SM-PDS $\mathcal{P}$ satisfies a BranchCARET formula ψ , we follow the lines of [21] and [16]. We construct a kind of alternating Büchi pushdown system $\mathcal{BP}$, such that there is an infinite run in $\mathcal{BP}$ that visits accepting control locations infinitely many times only if $\mathcal{P}$ satisfies ψ . This is not trivial for two reasons. First, BranchCARET can reason about *abstract* successors, which refer to the next step in the *same function*. I.e., if the current step is a call, then the *abstract* successor ‘steps over’ the inner call and refers directly to the return location. Although this problem was solved for standard non self-modifying PDS, the problem is harder for SM-PDS. Not only one needs to know the return location, but also, if a self-modification happened during the inner call, then the return location must reflect the self-modification. The second reason is the *caller* successor in BranchCARET. This kind of successors refers to the latest call *in the past*. Again, to reason about the past state, one needs to know if a self-modification happened since this past state. We solve these two problems by introducing new types of control locations that are activated when reasoning about abstract and caller successors. Moreover, we store information about states in the past inside of special stack symbols that we call *checkpoints*. If ψ requires to ‘go back in time’, these checkpoints can be read from the stack and the old state can be restored. Then, to find accepting runs in $\mathcal{BP}$, we introduce an efficient algorithm that constructs a finite-state automaton $\mathcal{A}$ that accepts configurations from which an accepting run can be constructed.

We have implemented our algorithm in a tool for model checking binary programs. Then, we expressed several malicious behaviour using BranchCARET formulas, such that if a given binary satisfies the formula, then the binary is marked as malware. We then applied the tool on malwares taken from public databases and the tool was able to detect malicious behaviour in many self-modifying malwares.

Related Works Model checking of binary code is an extensively studied field with many approaches. One of the most popular approaches is symbolic execution [18]. Symbolic

execution of binaries has been employed in such platforms as BINSEC [14], S2E [11], and Zorya [15]. However, symbolic execution alone is not enough for malware detection, as one needs a robust way to specify features of malware.

One way of describing malicious behaviour is temporal logic, which was used by numerous studies [4]–[6], [17], [20], [23] in the context of malware detection. However, these works do not consider self-modification.

Some studies [7], [9], [10], [12] proposed formal models for self-modifying code, but such models do not scale to large real world binaries. Control flow reconstruction of self-modifying binaries was proposed by [3], but this technique ignores execution stack. Other studies [8], [19] provide a technique to disassemble packed binaries. However, these works focus solely on packing behaviour, and not on self-modification in general. [24] proposed model checking for Self-Modifying Pushdown System (SM-PDS). However, in these works, only reachability, LTL, and CTL model checking is considered. Whereas our approach adopts the SM-PDS model and proposes effective model checking of SM-PDS over BranchCARET.

Outline Section II introduces base concepts used throughout this paper. Section III presents the main contribution - direct and efficient BranchCARET model checking algorithm of SM-PDS. In Section IV, we describe the results of practical experiments, including malware detection using BranchCARET model checking. For lack of space, proofs can be found in the Appendix.

II. PRELIMINARIES

A. Self-Modifying Pushdown Systems

We define our model for self-modifying binary code:

Definition 1. A *Labelled Self-Modifying Pushdown System (SM-PDS)* is a tuple $\mathcal{P} = (P, \Gamma, \Delta)$, such that P is a finite set of control locations, Γ is a stack alphabet, Δ is a finite set of rules of the forms: (1) $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w$, (2) $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2$, and (3) $p\gamma \xrightarrow{(\rho, \rho')}_{ret} p'$, where $p, p' \in P$, $\gamma, \gamma_1, \gamma_2 \in \Gamma$, $w \in \Gamma^*$, and $\rho, \rho' \subseteq \Delta$.

To express the self-modifying behaviour of SM-PDS, we introduce the notion of *phase* $\theta \subseteq \Delta$, which is the current set of rules that can be applied. Intuitively, a rule of the form $r = p\gamma \xrightarrow{(\rho, \rho')}_t p'w$ for $t \in \{call, int, ret\}$, can be applied only if the current phase θ contains r and every rule from ρ . Such rule pops γ from the stack, pushes the word w onto the stack (w can be empty), changes the current control location from p to p' , removes every rule in ρ from θ , and adds every rule from ρ' to θ . Hence, a rule of the form $p\gamma \xrightarrow{(\emptyset, \emptyset)}_t p'w$ is a standard PDS rule that does not perform self-modification.

A rule of the form $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2$ often corresponds to a call statement of the form $\gamma \xrightarrow{call \text{ proc}} \gamma_2$, where *proc* is the name of the called procedure, γ is the location of the call statement, γ_1 is the entry location of *proc*, γ_2 is the return

location, p and p' represent global state of the program, which might contain global variables and return values. A rule of the form $p\gamma \xrightarrow{(\rho, \rho')}_{ret} p'$ corresponds to a return statement, and a rule of the form $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w$ corresponds to an internal statement within a procedure.

For an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta)$, a *configuration* $c \in P \times \Gamma^* \times 2^\Delta$ of $\mathcal{P}$ is a tuple of the form $(\langle p, \omega \rangle, \theta)$, where $p \in P$ is the current control location, $\omega \in \Gamma^*$ is the current stack content, and $\theta \subseteq \Delta$ is the current phase. Let $Conf$ be the set of all possible configurations of $\mathcal{P}$. For configurations c, c' , and a label $t \in \{call, int, ret\}$, the successor relation $c \xrightarrow{t} c'$ is defined as follows. For any $t \in \{call, int, ret\}$, $p, p' \in P$, $\omega \in \Gamma^*$, $\theta, \rho, \rho' \subseteq \Delta$, and $\theta' = (\theta \setminus \rho) \cup \rho'$:

- if $p\gamma \xrightarrow{(\rho, \rho')}_t p'w \in \theta$ and $\rho \subseteq \theta$, then $(\langle p, \gamma\omega \rangle, \theta) \xrightarrow{t} (\langle p', w\omega \rangle, \theta')$.

A *run* π on $\mathcal{P}$ is an infinite sequence of the form $\pi = (c_0, t_0)(c_1, t_1) \dots$, where for every $i \geq 0$, $c_i \in Conf$, and $\exists t_i \in \{call, int, ret\}$, s.t. $c_i \xrightarrow{t_i} c_{i+1}$. We denote by $\pi(i)$ the i -th configuration c_i of π . We say that π starts with c_0 . Let $Ext(\pi, i)$ be the set of runs on $\mathcal{P}$, s.t. for each $\pi' \in Ext(\pi, i)$, π' is an extension of π from i , i.e. for each $j \leq i$, $\pi'(j) = \pi(j)$.

Let $\pi = (c_0, t_0)(c_1, t_1) \dots$ be a run on $\mathcal{P}$, such that for i , $i \geq 0$, $\pi(i) = c_i = (\langle p_i, \omega_i \rangle, \theta_i)$, where $p_i \in P$, $\omega_i \in \Gamma^*$, $\theta_i \subseteq \Delta$. Each index $i \geq 0$ on π may have three types of successors:

- the *global successor*, $next_\pi^g(i) = i + 1$;
- the *abstract successor*, $next_\pi^a(i)$, depends on the transition $c_i \xrightarrow{t_i} c_{i+1}$:
 - if $t_i = call$, then $next_\pi^a(i)$ is the index of the matching *ret*,
 - if $t_i = int$, then $next_\pi^a(i) = i + 1$, and
 - if $t_i = ret$, then $next_\pi^a(i) = \perp$;
- the *caller successor*, $next_\pi^c(i)$, is the index of the latest unmatched *call* in the past ($next_\pi^c(i) < i$), otherwise $next_\pi^c(i) = \perp$.

Intuitively, a *global* successor is a “standard successor”. An *abstract* successor depends on the nature of the transition $c_i \xrightarrow{t_i} c_{i+1}$. If it is a *call*, then the abstract successor is the corresponding return location. If it is an internal transition, then the abstract successor is the global successor. If it is a return, then the abstract successor does not exist. The *caller* successor corresponds to the call site of the current procedure. Note that sometimes $\pi(i)$ might not have an abstract (resp. caller) successor. In these cases, we say that $next_\pi^a(i)$ (resp. $next_\pi^c(i)$) is undefined, or $next_\pi^a(i) = \perp$ (resp. $next_\pi^c(i) = \perp$).

B. BranchCARET formulas

Given a set of atomic propositions AP , we define a BranchCARET formula ψ over AP as follows:

Definition 2. A BranchCARET formula ψ is ($a \in AP$ and $b \in \{g, a, c\}$):

$$\begin{aligned} \psi ::= & a \mid \neg a \mid \psi \vee \psi \mid \psi \wedge \psi \mid EX^b \psi \mid AX^b \psi \\ & \mid E[\psi U^b \psi] \mid A[\psi U^b \psi] \mid E[\psi R^b \psi] \mid A[\psi R^b \psi] \end{aligned}$$

For an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta)$ and a BranchCARET formula ψ , let λ be a labelling function $\lambda : P \rightarrow 2^{AP}$ that labels each control location with a set of atomic propositions. Consider a run $\pi = (c_0, t_0)(c_1, t_1) \dots$ on $\mathcal{P}$, s.t. for $i \geq 0$, $c_i = (\langle p_i, w_i \rangle, \theta_i)$. We write $(\pi, i) \models \psi$ if π satisfies ψ at the i -th step:

- 1) $(\pi, i) \models e$, where $e \in AP$, if $e \in \lambda(p_i)$;
- 2) $(\pi, i) \models \neg e$, where $e \in AP$, if $e \notin \lambda(p_i)$;
- 3) $(\pi, i) \models \psi_1 \vee \psi_2$ if $(\pi, i) \models \psi_1$ or $(\pi, i) \models \psi_2$;
- 4) $(\pi, i) \models \psi_1 \wedge \psi_2$ if $(\pi, i) \models \psi_1$ and $(\pi, i) \models \psi_2$;
- 5) $(\pi, i) \models EX^b \psi$, where $b \in \{g, a, c\}$, if there exists $\pi' \in Ext(\pi, i)$, s.t. $next_\pi^b(i) \neq \perp$ and $(\pi', next_\pi^b(i)) \models \psi$;
- 6) $(\pi, i) \models AX^b \psi$, where $b \in \{g, a, c\}$, if for all $\pi' \in Ext(\pi, i)$, $next_\pi^b(i) \neq \perp$ implies $(\pi', next_\pi^b(i)) \models \psi$;
- 7) $(\pi, i) \models E[\psi_1 U^b \psi_2]$, where $b \in \{g, a, c\}$, if there exists $\pi' \in Ext(\pi, i)$, s.t. there is a sequence $s = s_0 s_1 \dots s_k$, where $s_0 = i$, for j , $0 \leq j < k$, $s_{j+1} = next_{\pi'}^b(s_j)$, $(\pi', s_j) \models \psi_1$, and $(\pi', s_k) \models \psi_2$;
- 8) $(\pi, i) \models A[\psi_1 U^b \psi_2]$, where $b \in \{g, a, c\}$, if for all $\pi' \in Ext(\pi, i)$, there is a sequence $s = s_0 s_1 \dots s_k$, where $s_0 = i$, for j , $0 \leq j < k$, $s_{j+1} = next_{\pi'}^b(s_j)$, $(\pi', s_j) \models \psi_1$, and $(\pi', s_k) \models \psi_2$;
- 9) $(\pi, i) \models E[\psi_1 R^b \psi_2]$, where $b \in \{g, a, c\}$, if there exists $\pi' \in Ext(\pi, i)$ and a sequence $s = s_0 s_1 \dots$, where $s_0 = i$ and for j , $j \geq 0$, $s_{j+1} = next_{\pi'}^b(s_j)$, we have that $(\pi', s_j) \models \psi_1$ implies that there is $k < j$, s.t. $(\pi', s_k) \models \psi_2$;
- 10) $(\pi, i) \models A[\psi_1 R^b \psi_2]$, where $b \in \{g, a, c\}$, if for all $\pi' \in Ext(\pi, i)$, there is a sequence $s = s_0 s_1 \dots$, s.t. $s_0 = i$ and for j , $j \geq 0$, $s_{j+1} = next_{\pi'}^b(s_j)$, we have that $(\pi', s_j) \models \psi_1$ implies that there is $k < j$, s.t. $(\pi', s_k) \models \psi_2$.

We say that $\pi \models \psi$ if $(\pi, 0) \models \psi$. For a configuration c of $\mathcal{P}$, $c \models \psi$ iff there exists a run π on $\mathcal{P}$ starting from c , s.t. $\pi \models \psi$. Let $Cl(\psi)$ be the closure of ψ , i.e. the set of all subformulas of ψ .

Intuitively, EX^g and AX^g correspond to *global* successors, EX^a and AX^a correspond to *abstract* successors, EX^c and AX^c correspond to *caller* successors. $E[\psi U^b \psi]$ and $A[\psi U^b \psi]$ are *until* operators. $E[\psi R^b \psi]$ and $A[\psi R^b \psi]$ are *release* operators. A BranchCARET formula with only operators labelled by g is equivalent to a CTL formula.

III. BRANCHCARET MODEL CHECKING OF SM-PDSS

This section presents our contribution. We propose a direct algorithm for model checking SM-PDSSs against BranchCARET formulas. We fix an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta)$ and a BranchCARET formula ψ defined over a set of atomic propositions AP with a labelling function λ .

A. Self-Modifying Alternating Büchi Pushdown System with Checkpoints

BranchCARET model checking introduces a significant complication in the model checking process - a formula can talk about *caller successors*. Since caller successors are in the past of the current configuration, one needs a way to obtain previous states from the current configuration. To store information about previous states, we introduce a notion of *checkpoints*, which are a special kind of stack symbols. From now on, checkpoints will be denoted as forms of χ . We define Self-Modifying Alternating Büchi Pushdown Systems with Checkpoints:

Definition 3. A Self-Modifying Alternating Büchi Pushdown System with Checkpoints (SM-ABPDS) is a tuple $\mathcal{BP} = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, \delta, F)$, where P' is a finite set of control locations, Γ' is a stack alphabet, X is a checkpoint alphabet, Δ' is a set of alternating rules, *restore* is a set of rules that restore phase from a checkpoint, *discard* is a set of rules that discard phases from checkpoints, δ is a phase alphabet, and F is an accepting set of control locations.

Rules in Δ' have the following form: $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\}$, where $p \in P'$, $\gamma \in \Gamma'$, for $1 \leq i \leq h$, σ_i has the form $(\rho_i, \rho'_i) \in 2^\delta \times 2^\delta$, $p_i \in P'$, and either $w_i \in \Gamma'^*$ or $w_i \in \Gamma' \times X$.

Rules in *restore* and *discard* have the form $p\chi \hookrightarrow p'\gamma$, where $p, p' \in P'$, $\chi \in X$, and $\gamma \in \Gamma'$.

A phase θ of $\mathcal{BP}$ is a set $\theta \subseteq \delta$ that consists of symbols from the phase alphabet δ . A checkpoint χ , when on the stack, is always paired with a phase θ in a symbol of the form χ^θ . The set of possible symbols on the stack of $\mathcal{BP}$ is $G = \Gamma' \cup (X \times 2^\delta)$. Let us define a function $em : 2^\delta \times (\Gamma'^* \cup (\Gamma' \times X)) \rightarrow G^*$, defined as $em(\theta, w) = w$ for $w \in \Gamma'^*$, and $em(\theta, \gamma\chi) = \gamma\chi^\theta$ for $\gamma\chi \in \Gamma' \times X$. The em function embeds the phase θ into the checkpoints of the word w located on the right hand side of the rules in Δ' . Formally, a configuration of $\mathcal{BP}$ is a tuple $(\langle p, w \rangle, \theta) \in P \times G^* \times 2^\delta$. Let the set of all configurations of $\mathcal{BP}$ be $Conf_{\mathcal{BP}}$. For a configuration c and set of configurations $S \subseteq Conf_{\mathcal{BP}}$, we define the successor relation $c \Rightarrow S$:

- 1) if $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\} \in \Delta'$, where for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$, then $(\langle p, \gamma\omega \rangle, \theta) \Rightarrow \{(\langle p_i, em(\theta, w_i)\omega \rangle, (\theta \setminus \rho_i) \cup \rho'_i) \mid \text{if } \rho_i \subseteq \theta\}$;
- 2) if $p\chi \hookrightarrow p'\gamma \in \text{restore}$, then $(\langle p, \chi^\theta\omega \rangle, \theta') \Rightarrow \{(\langle p', \gamma\omega \rangle, \theta')\}$ for any $\theta' \subseteq \delta$;
- 3) if $p\chi \hookrightarrow p'\gamma \in \text{discard}$, then $(\langle p, \chi^\theta\omega \rangle, \theta') \Rightarrow \{(\langle p', \gamma\omega \rangle, \theta')\}$ for any $\theta' \subseteq \delta$;

Consider an SM-ABPDS $\mathcal{BP} = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, \delta, F)$. The idea behind $\mathcal{BP}$ is that the execution of $\mathcal{BP}$ should correspond to the execution of $\mathcal{P}$. To do so, $\mathcal{BP}$ needs information about the phase in $\mathcal{P}$. However, since the phase in $\mathcal{P}$ consists of rules from Δ , and $\mathcal{BP}$ has a different set of rules Δ' , then $\mathcal{BP}$

needs to synchronise with the current phase of $\mathcal{P}$. To this aim, we introduce the *phase alphabet* δ , which represents symbols that can be in the phase of $\mathcal{BP}$. Later, δ will correspond to Δ in $\mathcal{P}$ and the phase $\theta \subseteq \delta$ of $\mathcal{BP}$ will then correspond directly to the phase $\theta \subseteq \Delta$ of $\mathcal{P}$.

Intuitively, given a configuration $(\langle p, \gamma\omega \rangle, \theta)$ of $\mathcal{BP}$, application of a rule of the form $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\} \in \Delta'$ corresponds to applying several SM-PDS rules of the forms $p\gamma \xrightarrow{(\rho_i, \rho'_i)} p_iw_i$ for $1 \leq i \leq h$, where $\sigma_i = (\rho_i, \rho'_i)$, at the same time. $\mathcal{BP}$ transitions into several configurations at once (it is alternating) and the current configuration has a *set* of successors. We say that such rule allows h possible transitions. For $1 \leq i \leq h$, the label $\sigma_i = (\rho_i, \rho'_i)$ specifies (1) which rules must be in the current phase for the i -th transition, and (2) the self-modification performed by the i -th transition. The i -th transition takes place only if $\rho_i \subseteq \theta$, and it changes the control location from p to p_i , pops γ from the stack, removes every rule in ρ_i from θ , and adds every rule from ρ'_i to θ . Then it pushes a word $w'_i \in G^*$ onto the stack. w'_i depends on w_i , which can be of two forms:

- if $w_i \in \Gamma'^*$ (it does not contain checkpoints), then $w_i = w'_i$,
- if $w_i = \gamma'\chi \in \Gamma' \times X$, where χ is a checkpoint, then $w'_i = \gamma'\chi^\theta$.

Therefore, the i -th successor is a configuration $(\langle p_i, w'_i\omega \rangle, \theta_i)$, where $\theta_i = (\theta \setminus \rho_i) \cup \rho'_i$. If, for some i , $\sigma_i = (\emptyset, \emptyset)$, then, this means that the current configuration can perform the i -th transition independent of its phase. In that case, for succinctness, we say that $\sigma_i = \top$.

Once a checkpoint χ^θ is pushed on the stack, it cannot be popped using rules from Δ' because they require a symbol $\gamma \in \Gamma'$ at the top of the stack. We introduce *restore* and *discard* rules that deal with checkpoints. A *restore* rule of the form $p\chi \xrightarrow{\text{restore}} p'\gamma$ pops χ^θ from the stack for any θ , sets the current phase to θ , changes the control location from p to p' , and pushes γ onto the stack. A *discard* rule of the form $p\chi \xrightarrow{\text{discard}} p'\gamma$ pops χ^θ from the stack for any θ , changes the control location from p to p' , and pushes γ onto the stack (without changing the current phase).

A run η on $\mathcal{BP}$ starting from configuration $c \in Conf_{\mathcal{BP}}$ is a tree with each node labelled by a configuration of $\mathcal{BP}$. The root is labelled with c , and a node labelled with c' has the set of children S if $c' \Rightarrow S$ for some $S \subseteq Conf_{\mathcal{BP}}$. A run η on $\mathcal{BP}$ is *accepting* if every branch of η visits accepting control locations from F infinitely many times.

B. From BranchCARET Model Checking Problem to the Emptiness Problem of SM-ABPDS

Let $Cl^a(\psi) = \{QX^a\phi_1, Q[\phi_1U^a\phi_2], Q[\phi_1R^a\phi_2] \in Cl(\psi) \mid Q \in \{E, A\}\}$ and $Cl^c(\psi) = \{QX^c\phi_1, Q[\phi_1U^c\phi_2], Q[\phi_1R^c\phi_2] \in Cl(\psi) \mid Q \in \{E, A\}\}$. We can construct SM-ABPDS $\mathcal{BP} = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, \Delta, F)$, where:

- $P' = P'_g \cup P'_a \cup P'_c$, where:

- $P'_g = P \times Cl(\psi)$ is a set of control locations of the form $[p, \phi]$,
- $P'_a = P \times Cl^a(\psi) \times \{r\}$ is a set of control locations of the form $[p, \phi]^r$,
- $P'_c = \{p^c\} \times Cl^c(\psi)$ is a set of control locations of the form $[p^c, \phi]$, s.t. p^c is a new state;
- $\Gamma' = \Gamma \cup (\Gamma \times \{r\})$, s.t. $\Gamma \times \{r\}$ is the set of stack symbols of the form γ^r ;
- $X = \Gamma \times P \times \Gamma$;
- $F = \{[p, e], [p, \neg e] \mid e \in AP\} \cup F^g \cup F^a \cup F^c$, where:
 - $F^g = \{[p, \phi] \in P'_g \mid \phi = Q[\phi_1 R^g \phi_2] \text{ for } Q \in \{E, A\}\}$,
 - $F^a = \{[p, \phi]^r \in P'_a \mid \phi = Q[\phi_1 R^a \phi_2] \text{ or } \phi = AX^a \phi_1 \text{ for } Q \in \{E, A\}\}$,
 - $F^c = \{[p^c, \phi] \in P'_c \mid \phi = Q[\phi_1 R^c \phi_2] \text{ or } \phi = AX^c \phi_1 \text{ for } Q \in \{E, A\}\}$.

Without loss of generality, we assume that Γ contains a symbol $\#$ that is always located at the bottom of the stack and can never be popped. Moreover, for any rule of the form $r = p\gamma \xrightarrow{(\rho, \rho')}_t p'w \in \Delta$, w.l.o.g. we assume that $r \in \rho$. We can assume this because any rule $r = p\gamma \xrightarrow{(\rho, \rho')}_t p'w \in \Delta$, where $r \notin \rho$, can be effectively translated to an equivalent rule of the form $p\gamma \xrightarrow{(\rho \cup \{r\}, \rho' \cup \{r\})}_t p'w$. Therefore, for $r = p\gamma \xrightarrow{(\rho \cup \{r\}, \rho' \cup \{r\})}_t p'w \in \Delta$ to be applied at the phase θ , it is sufficient to assure that $\rho \cup \{r\} \subseteq \theta$.

We denote by *True* the atomic proposition that is satisfied by every control location in P , and we denote by *False* the atomic proposition that is never satisfied. The SM-ABPDS rules Δ' , *restore*, and *discard* are defined as follows. For $p \in P$, $\gamma \in \Gamma$, $\phi \in Cl(\psi)$:

- 1) if $\phi = a \in AP$ and $a \in \lambda(p)$, then $[p, a]\gamma \xrightarrow{[\top]} \{[p, a]\gamma\}$;
- 2) if $\phi = \neg a \in AP$ and $a \notin \lambda(p)$, then $[p, \neg a]\gamma \xrightarrow{[\top]} \{[p, \neg a]\gamma\}$;
- 3) if $\phi = \phi_1 \vee \phi_2$, then $[p, \phi]\gamma \xrightarrow{[\top]} \{[p, \phi_1]\gamma\}$ and $[p, \phi]\gamma \xrightarrow{[\top]} \{[p, \phi_2]\gamma\} \in \Delta'$;
- 4) if $\phi = \phi_1 \wedge \phi_2$, then $[p, \phi_1 \wedge \phi_2]\gamma \xrightarrow{[\top, \top]} \{[p, \phi_1]\gamma, [p, \phi_2]\gamma\}$;
- 5) if $\phi = EX^g \phi'$, then $[p, EX^g \phi']\gamma \xrightarrow{[\sigma]} \{[p', \phi']w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \Delta$, $p\gamma \xrightarrow{(\rho, \rho')}_{ret} p'w \in \Delta$, or $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2 \in \Delta$ and $w = \gamma_1[\gamma_2, p, \gamma]$;
- 6) if $\phi = AX^g \phi'$, then $[p, AX^g \phi']\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{[p_1, \phi']w_1, \dots, [p_h, \phi']w_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$, if $t_i \in \{int, ret\}$, then $w_i = w'_i$, and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $w_i = \gamma_1[\gamma_2, p, \gamma]$;
- 7) if $\phi = EX^a \phi'$, then $[p, EX^a \phi']\gamma \xrightarrow{[(\rho, \rho')]} \{p''w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \Delta$ and $p'' = [p', \phi']$, or

- 8) if $\phi = EX^a \phi_1$, $\phi = E[\phi_1 U^a \phi_2]$, or $\phi = E[\phi_1 R^a \phi_2]$, then $[p, \phi]^r\gamma \xrightarrow{[(\rho, \rho')]} \{[p', \phi]^r w\} \in \Delta'$ for each $p\gamma \xrightarrow{(\rho, \rho')}_t p'w \in \Delta$;
- 9) if $\phi = AX^a \phi'$, then $[p, AX^a \phi']\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p'_1w_1, \dots, p'_hw_h\} \in \Delta'$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$; if $t_i = int$, then $p'_i = [p_i, \phi']$ and $w_i = w'_i$; if $t_i = ret$, then $p'_i = [p_i, True]$ and $w_i = w'_i = \varepsilon$; and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $p'_i = [p_i, \phi']^r$ and $w_i = \gamma_1\gamma_2^r$;
- 10) if $\phi = AX^a \phi_1$, $\phi = A[\phi_1 U^a \phi_2]$, or $\phi = A[\phi_1 R^a \phi_2]$, then $[p, \phi]^r\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{[p_1, \phi]^r w_1, \dots, [p_h, \phi]^r w_h\} \in \Delta'$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_t p_1w_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_t p_hw_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, where for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$;
- 11) if $\phi = QX^a \phi_1$ for $Q \in \{E, A\}$, then $[p, \phi]^r\gamma^r \xrightarrow{[\top]} \{[p, \phi_1]\gamma\} \in \Delta'$;
- 12) if $\phi = Q[\phi_1 U^b \phi_2]$ for $Q \in \{E, A\}$ and $b \in \{g, a, c\}$, then $[p, \phi]\gamma \xrightarrow{[\top]} \{[p, \phi_2]\gamma\}$;
- 13) if $\phi = Q[\phi_1 R^b \phi_2]$ for $Q \in \{E, A\}$ and $b \in \{g, a, c\}$, then $[p, \phi]\gamma \xrightarrow{[\top, \top]} \{[p, \phi_1]\gamma, [p, \phi_2]\gamma\}$;
- 14) if $\phi = E[\phi_1 U^g \phi_2]$, then $[p, \phi]\gamma \xrightarrow{[(\rho, \rho), (\rho, \rho')]} \{[p, \phi_1]\gamma, [p', \phi]w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \Delta$, $p\gamma \xrightarrow{(\rho, \rho')}_{ret} p'w \in \Delta$, or $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2 \in \Delta$ and $w = \gamma_1[\gamma_2, p, \gamma]$;
- 15) if $\phi = A[\phi_1 U^g \phi_2]$, then $[p, \phi]\gamma \xrightarrow{[\top, \sigma_1, \dots, \sigma_h]} \{[p, \phi_1]\gamma, [p_1, \phi]w_1, \dots, [p_h, \phi]w_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$, if $t_i \in \{int, ret\}$, then $w_i = w'_i$, and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $w_i = \gamma_1[\gamma_2, p, \gamma]$;
- 16) if $\phi = E[\phi_1 U^a \phi_2]$, then $[p, \phi]\gamma \xrightarrow{[(\rho, \rho), (\rho, \rho')]} \{[p, \phi_1]\gamma, p''w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \Delta$ and $p'' = [p', \phi]$, or $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2 \in \Delta$, $p'' = [p', \phi]^r$, and $w = \gamma_1\gamma_2^r$;
- 17) if $\phi = A[\phi_1 U^a \phi_2]$, then $[p, \phi]\gamma \xrightarrow{[\top, \sigma_1, \dots, \sigma_h]} \{[p, \phi_1]\gamma, p'_1w_1, \dots, p'_hw_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$; if $t_i = int$, then $p'_i = [p_i, \phi]$ and $w_i = w'_i$; if $t_i = ret$, then $p'_i = [p_i, False]$ and $w_i = w'_i = \varepsilon$; and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $p'_i = [p_i, \phi]^r$ and $w_i = \gamma_1\gamma_2^r$;
- 18) if $\phi = Q[\phi_1 U^a \phi_2]$ or $\phi = Q[\phi_1 R^a \phi_2]$ for $Q \in \{E, A\}$, then $[p, \phi]^r\gamma^r \xrightarrow{[\top]} \{[p, \phi]\gamma\} \in \Delta'$;

- 19) if $\phi = E[\phi_1 R^g \phi_2]$, then $[p, \phi] \gamma \xrightarrow{[(\rho, \rho'), (\rho, \rho')]} \{[p, \phi_2] \gamma, [p, \phi] w\}$, s.t. either $p \gamma \xrightarrow{(\rho, \rho')}_{int} p' w \in \Delta$, $p \gamma \xrightarrow{(\rho, \rho')}_{ret} p' w \in \Delta$, or $p \gamma \xrightarrow{(\rho, \rho')}_{call} p' \gamma_1 \gamma_2 \in \Delta$ and $w = \gamma_1 [\gamma_2, p, \gamma]$;
- 20) if $\phi = A[\phi_1 R^g \phi_2]$, then $[p, \phi] \gamma \xrightarrow{[\top, \sigma_1, \dots, \sigma_h]} \{[p, \phi_2] \gamma, [p_1, \phi] w_1, \dots, [p_h, \phi] w_h\}$, s.t. $\{p \gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1 w'_1, \dots, p \gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_h w'_h\}$ is the set of rules in Δ with $p \gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$, if $t_i \in \{int, ret\}$, then $w_i = w'_i$, and if $t_i = call$ and $w'_i = \gamma_1 \gamma_2$, then $w_i = \gamma_1 [\gamma_2, p, \gamma]$;
- 21) if $\phi = E[\phi_1 R^a \phi_2]$, then $[p, \phi] \gamma \xrightarrow{[(\rho, \rho'), (\rho, \rho')]} \{[p, \phi_2] \gamma, p'' w\}$, s.t. either $p \gamma \xrightarrow{(\rho, \rho')}_{int} p' w \in \Delta$ and $p'' = [p', \phi]$, or $p \gamma \xrightarrow{(\rho, \rho')}_{ret} p' w \in \Delta$ and $p'' = [p', True]$, or $p \gamma \xrightarrow{(\rho, \rho')}_{call} p' \gamma_1 \gamma_2 \in \Delta$, $p'' = [p', \phi]^r$, and $w = \gamma_1 \gamma_2^r$;
- 22) if $\phi = A[\phi_1 R^a \phi_2]$, then $[p, \phi] \gamma \xrightarrow{[\top, \sigma_1, \dots, \sigma_h]} \{[p, \phi_2] \gamma, p'_1 w_1, \dots, p'_h w_h\}$, s.t. $\{p \gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1 w'_1, \dots, p \gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_h w'_h\}$ is the set of rules in Δ with $p \gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$; if $t_i = int$, then $p'_i = [p_i, \phi]$ and $w_i = w'_i$; if $t_i = ret$, then $p'_i = [p_i, True]$ and $w_i = w'_i = \varepsilon$; and if $t_i = call$ and $w'_i = \gamma_1 \gamma_2$, then $p'_i = [p_i, \phi]^r$ and $w_i = \gamma_1 \gamma_2^r$;
- 23) if $\phi = QX^c \phi'$, where $Q \in \{E, A\}$, then $[p, \phi] \gamma \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$;
- 24) if $\phi = Q[\phi_1 U^c \phi_2]$, where $Q \in \{E, A\}$, then $[p, \phi] \gamma \xrightarrow{[\top, \top]} \{[p, \phi_1] \gamma, [p^c, \phi] \varepsilon\}$;
- 25) if $\phi = Q[\phi_1 R^c \phi_2]$, where $Q \in \{E, A\}$, then $[p, \phi] \gamma \xrightarrow{[\top, \top]} \{[p, \phi_2] \gamma, [p^c, \phi] \varepsilon\}$;
- 26) $[p^c, \phi] \gamma \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\} \in \Delta'$ for $\gamma \neq \#$ and $[p^c, \phi] \# \xrightarrow{[\top]} \{[p^c, \phi] \#\}$;
- 27) if $\phi = QX^c \phi_1$ for $Q \in \{E, A\}$, $[p^c, \phi] [\gamma, p_0, \gamma_0] \xrightarrow{restore} [p_0, \phi_1] \gamma_0 \in restore$ for $p_0 \in P$ and $\gamma_0 \in \Gamma$;
- 28) if $\phi = Q[\phi_1 U^c \phi_2]$ or $\phi = Q[\phi_1 R^c \phi_2]$ for $Q \in \{E, A\}$, $[p^c, \phi] [\gamma, p_0, \gamma_0] \xrightarrow{restore} [p_0, \phi] \gamma_0 \in restore$ for $p_0 \in P$ and $\gamma_0 \in \Gamma$;
- 29) $[p, \phi] [\gamma, p_0, \gamma_0] \xrightarrow{discard} [p, \phi] \gamma \in discard$ for $p \neq p^c$, $p_0 \in P$, and $\gamma_0 \in \Gamma$.

Roughly speaking, $\mathcal{BP}$ is a kind of product between $\mathcal{P}$ and ψ . We construct $\mathcal{BP}$ in such a way that for a configuration $(\langle p, w \rangle, \theta)$ of $\mathcal{P}$, $(\langle p, w \rangle, \theta) \models \psi$ iff there exists an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \psi], w \rangle, \theta)$. Let us explain the intuition behind this construction. Suppose that $(\langle p, \gamma \omega \rangle, \theta) \models \psi$, s.t. there should be an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \psi], \gamma \omega \rangle, \theta)$.

If $\psi = a \in AP$, then $p \in \lambda(a)$ and item 1 creates a rule of the form $[p, a] \gamma \xrightarrow{[\top]} \{[p, a] \gamma\}$, which creates an infinite loop

of the form $(\langle [p, a], \gamma \omega \rangle, \theta) \implies (\langle [p, a], \gamma \omega \rangle, \theta) \implies \dots$. And since $[p, a] \in F$, then this infinite loop constitutes an accepting run.

If $\psi = \neg a$, s.t. $a \in AP$, then $p \notin \lambda(a)$ and item 2 creates a rule of the form $[p, \neg a] \gamma \xrightarrow{[\top]} \{[p, \neg a] \gamma\}$, which creates an infinite loop of the form $(\langle [p, \neg a], \gamma \omega \rangle, \theta) \implies (\langle [p, \neg a], \gamma \omega \rangle, \theta) \implies \dots$. And since $[p, \neg a] \in F$, then this infinite loop constitutes an accepting run.

If $\psi = \phi_1 \vee \phi_2$, then either $(\langle p, \gamma \omega \rangle, \theta) \models \phi_1$, or $(\langle p, \gamma \omega \rangle, \theta) \models \phi_2$. Following the idea of the construction inductively, there should be an accepting run from either $(\langle [p, \phi_1], \gamma \omega \rangle, \theta)$ or $(\langle [p, \phi_2], \gamma \omega \rangle, \theta)$. Item 3 creates rules of the forms $[p, \psi] \gamma \xrightarrow{[\top]} \{[p, \phi_1] \gamma\}$ and $[p, \psi] \gamma \xrightarrow{[\top]} \{[p, \phi_2] \gamma\}$, which create transitions of the forms $(\langle [p, \psi], \gamma \omega \rangle, \theta) \implies (\langle [p, \phi_1], \gamma \omega \rangle, \theta)$ and $(\langle [p, \psi], \gamma \omega \rangle, \theta) \implies (\langle [p, \phi_2], \gamma \omega \rangle, \theta)$. Therefore, there is an accepting run from $(\langle [p, \psi], \gamma \omega \rangle, \theta)$ if there is an accepting run from either $(\langle [p, \phi_1], \gamma \omega \rangle, \theta)$, or from $(\langle [p, \phi_2], \gamma \omega \rangle, \theta)$.

If $\psi = \phi_1 \wedge \phi_2$, then $(\langle p, \gamma \omega \rangle, \theta) \models \phi_1$ and $(\langle p, \gamma \omega \rangle, \theta) \models \phi_2$. Following the idea of the construction inductively, there should be accepting runs from both $(\langle [p, \phi_1], \gamma \omega \rangle, \theta)$ and $(\langle [p, \phi_2], \gamma \omega \rangle, \theta)$. Item 4 creates a rule of the form $[p, \psi] \gamma \xrightarrow{[\top]} \{[p, \phi_1] \gamma, [p, \phi_2] \gamma\}$, which creates a transition of the form $(\langle [p, \psi], \gamma \omega \rangle, \theta) \implies \{(\langle [p, \phi_1], \gamma \omega \rangle, \theta), (\langle [p, \phi_2], \gamma \omega \rangle, \theta)\}$. Therefore, there is an accepting run from $(\langle [p, \psi], \gamma \omega \rangle, \theta)$ if there is an accepting run from both $(\langle [p, \phi_1], \gamma \omega \rangle, \theta)$ and $(\langle [p, \phi_2], \gamma \omega \rangle, \theta)$.

Now, let us demonstrate a more complex example. Suppose $AP = P$ and $\lambda(p) = \{p\}$ for any $p \in P$. Suppose we want to check if the configuration $(\langle p_1, \gamma_1 \# \rangle, \theta_1)$ in $\mathcal{P}$ satisfies a BranchCARET formula $\psi = A[\neg p_3 U^a p_2]$, which means that $\mathcal{P}$ does not visit p_3 before visiting p_2 in the same procedural context (i.e. this property should hold only in the current procedure). Let $\mathcal{P}$ be such that there are h rules with $p_1 \gamma_1$ on the left hand side: $\{p_1 \gamma_1 \xrightarrow{(\rho_1, \rho'_1)}_{int} p_2 \gamma_2, p_1 \gamma_1 \xrightarrow{(\rho_2, \rho'_2)}_{call} p_3 \gamma_3 \gamma_2, \dots, p_1 \gamma_1 \xrightarrow{(\rho_h, \rho'_h)}_{int} p^h w^h\}$. Out of these rules, suppose only $\rho_1, \rho_2 \subseteq \theta_1$, i.e. only the first two out of h rules can be applied. Note that the second rule is *call*, which means that to check $A[\neg p_3 U^a p_2]$, $\mathcal{BP}$ needs to find all possible corresponding returns. Then, item 17 will construct a rule of the form:

$$[p_1, \psi] \gamma_1 \xrightarrow{[\top, \sigma_1, \sigma_2, \dots, \sigma_h]} \{[p_1, \neg p_3] \gamma_1, [p_2, \psi] \gamma_2, [p_3, \psi]^r \gamma_3 \gamma_2^r, \dots, [p^h, \psi] w^h\}$$

Where $\sigma_i = (\rho_i, \rho'_i)$ for $1 \leq i \leq h$. Out of $h + 1$ possible transitions, only the first three can take place because $\emptyset, \rho_1, \rho_2 \subseteq \theta_1$. Let us construct the tree η starting from $(\langle [p_1, \psi], \gamma_1 \# \rangle, \theta_1)$. By applying the rule above, the root will have the following children: $(\langle [p_1, \neg p_3], \gamma_1 \# \rangle, \theta_1)$, $(\langle [p_2, \psi], \gamma_2 \# \rangle, \theta_2)$, and $(\langle [p_3, \psi]^r, \gamma_3 \gamma_2^r \# \rangle, \theta_3)$, s.t. $\theta_2 = (\theta_1 \setminus \rho_1) \cup \rho_1'$ and $\theta_3 = (\theta_1 \setminus \rho_2) \cup \rho_2'$. The first child corresponds to checking that $(\langle p_1, \gamma_1 \# \rangle, \theta_1) \models \neg p_3$. The

second child corresponds to checking if $(\langle p_2, \gamma_2 \# \rangle, \theta_2) \models \psi$, where $(\langle p_2, \gamma_2 \# \rangle, \theta_2)$ is the first possible successor of the initial configuration. The third child corresponds to checking if all possible returns from $(\langle p_3, \gamma_3 \gamma_2 \# \rangle, \theta_3)$ satisfy ψ . For the first child, item 2 constructs an infinite loop using the rule $[p_1, \neg p_3] \gamma_1 \xrightarrow{[\top]} \{[p_1, \neg p_3] \gamma_1\}$, which creates an accepting branch because $[p_1, \neg p_3] \in F$. Next, for the second child $(\langle p_2, \psi \rangle, \gamma_2 \# \rangle, \theta_2)$, item 12 creates a rule $[p_2, \psi] \gamma_2 \xrightarrow{[\top]} \{[p_2, p_2] \gamma_2\}$, thus obtaining the next configuration $(\langle p_2, p_2 \rangle, \gamma_2 \# \rangle, \theta_2)$. From there, item 1 constructs an accepting branch in the same way as item 2. Next let us consider the third child $(\langle p_3, \psi \rangle^r \gamma_3 \gamma_2 \# \rangle, \theta_3)$. Note that although $\mathcal{P}$ visits p_3 , this does not violate ψ , because this state happened inside the function call. Therefore, ψ needs to be checked at the return location. We use control locations of the form $[p, \psi]^r$ to ‘instruct’ $\mathcal{BP}$ to simulate the execution of $\mathcal{P}$ until the corresponding return. Since there may be several nested calls, we mark the return location γ_2^r with r to signify that when this symbol is on top of the stack, $\mathcal{BP}$ reached the corresponding return. Suppose that $p_3 \gamma_3 \xrightarrow{(\rho, \rho')^r}_{ret} p_2$ is the only rule with $p_3 \gamma_3$ as left hand side and $\rho \subseteq \theta_3$. Then, item 10 constructs a rule of the form $[p_3, \psi]^r \gamma_3 \xrightarrow{[(\rho, \rho')]} \{[p_2, \psi]^r \varepsilon\}$, s.t. the next configuration is $(\langle p_2, \psi \rangle^r, \gamma_2^r \# \rangle, \theta_4)$, where $\theta_4 = (\theta_3 \setminus \rho) \cup \rho'$. Then, only a rule constructed by item 18 can be applied: $[p_2, \psi]^r \gamma_2^r \xrightarrow{[\top]} \{[p_2, \psi] \gamma_2\}$, thus getting the next configuration $(\langle p_2, \psi \rangle, \gamma_2 \# \rangle, \theta_4)$. This configuration corresponds to finding if $(\langle p_2, \gamma_2 \# \rangle, \theta_4)$, which is the second abstract successor of the initial configuration, satisfies ψ . We can construct an accepting run from this configuration using the same reasoning as before (using rules created by items 12 and 1). Thus, all three branches of the constructed tree η are accepting and therefore, there is an accepting run starting from $(\langle p_1, \psi \rangle, \gamma_1 \# \rangle, \theta_1)$, which corresponds to $(\langle p_1, \gamma_1 \# \rangle, \theta_1) \models \psi$.

Next, let us consider a more complex example. Suppose we want to check if the configuration $(\langle p_1, \gamma_1 \# \rangle, \theta_1)$ satisfies $\psi = E[True \ U^g EX^c p_1]$, which means that eventually, $\mathcal{P}$ will call a function, and when the function is called, p_1 is the current control location. I.e., we need to check if there is an accepting run in $\mathcal{BP}$ starting from $(\langle p_1, \psi \rangle, \gamma_1 \# \rangle, \theta_1)$. Similar to the previous example, suppose there is a rule $p_1 \gamma_1 \xrightarrow{(\rho_2, \rho_2')^r}_{call} p_3 \gamma_3 \gamma_2 \in \Delta$ and $\rho_2 \subseteq \theta_1$. Then, item 14 creates the following rule: $[p_1, \psi] \gamma_1 \xrightarrow{[(\rho_2, \rho_2), (\rho_2, \rho_2')]} \{[p_1, True] \gamma_1, [p_3, \psi] \gamma_3 [\gamma_2, p_1, \gamma_1]\}$. Note that for the first possible transition, the label is (ρ_2, ρ_2) , i.e. this transition requires ρ_2 to be in the current phase but does not modify the phase. On the other hand, the second possible transition is labelled with (ρ_2, ρ_2') , which directly corresponds to the original rule in $\mathcal{P}$. The reason why the first label cannot be $\top$ is because otherwise, $\mathcal{BP}$ would take the first transition even when $\rho_2 \not\subseteq \theta_1$ (and thus failing to check the successor). This rule produces two children: $(\langle p_1, True \rangle, \gamma_1 \# \rangle, \theta_1)$ and $(\langle p_3, \psi \rangle, \gamma_3 [\gamma_2, p_1, \gamma_1]^{\theta_1} \# \rangle, \theta_3)$, where $\theta_3 = (\theta_1 \setminus \rho_2) \cup \rho_2'$. The first child is trivial. The second child corresponds to

checking if $(\langle p_3, \gamma_3 \gamma_2 \# \rangle, \theta_3) \models \psi$. Item 12 creates rule $[p_3, \psi] \gamma_3 \xrightarrow{[\top]} \{[p_3, EX^c p_1] \gamma_3\}$, which allows the next configuration to be $(\langle p_3, EX^c p_1 \rangle, \gamma_3 [\gamma_2, p_1, \gamma_1]^{\theta_1} \# \rangle, \theta_3)$. From there, only rule created by item 26 can be applied: $[p_3, EX^c p_1] \xrightarrow{[\top]} \{[p^c, EX^c p_1] \varepsilon\}$, which makes the next configuration to be $(\langle p^c, EX^c p_1 \rangle [\gamma_2, p_1, \gamma_1]^{\theta_1} \# \rangle, \theta_3)$. Next, only rule created by item 27 can be applied: $[p^c, EX^c p_1] [\gamma_2, p_1, \gamma_1] \xrightarrow{restore} [p_1, p_1] \gamma_1$, which results in the configuration $(\langle p_1, p_1 \rangle, \gamma_1 \# \rangle, \theta_1)$, from which there is an accepting run. Note that when $\mathcal{BP}$ executes the *restore* rule, the phase changes from θ_3 to θ_1 , which is the old phase. This is important for more complex formulas that require further execution from $(\langle p_1, \gamma_1 \# \rangle, \theta_1)$. Thus, there is an accepting run starting from $(\langle p_1, \psi \rangle, \gamma_1 \# \rangle, \theta_1)$, which corresponds to $(\langle p_1, \gamma_1 \# \rangle, \theta_1) \models \psi$.

Thus, we can show that (the proof can be found in Appendix A):

Theorem 1. *For a configuration $(\langle p, w \rangle, \theta)$ of $\mathcal{P}$ and a BranchCARET formula ψ , $(\langle p, w \rangle, \theta) \models \psi$ iff there is an accepting run in $\mathcal{BP}$ starting from $(\langle p, \psi \rangle, w, \theta)$.*

C. Solving the Emptiness Problem of SM-ABPDS with Check-points

To find accepting runs in $\mathcal{BP}$, we need to define functions $pre^+, pre^* : 2^{Conf_{\mathcal{BP}}} \rightarrow 2^{Conf_{\mathcal{BP}}}$, which compute the transitive and reflexive-transitive sets of predecessors respectively. The functions are defined as $pre^*(W) = \{c \mid c \in W \text{ or } c \implies S, \text{ s.t. } S \subseteq pre^*(W)\}$, and $pre^+(W) = \{c \mid c \implies S, \text{ s.t. } S \subseteq pre^*(W)\}$. Let $Y_0 = Conf_{\mathcal{BP}}$ be the set of configurations from which the accepting control locations are visited at least 0 times. Then, for $j \leq 0$, let $Y_{j+1} = pre^+(Y_j \cap (F \times G^* \times 2^\delta))$ be the set of configurations from which the accepting control locations are visited at least j times. Then, we can find the set of configurations $Y_{\mathcal{BP}}$, from which accepting control locations are visited infinitely many times by defining $Y_{\mathcal{BP}} = \bigcap_{j=0} Y_j$. We can show that (the proof can be found in Appendix B):

Theorem 2. *For any configuration c of $\mathcal{BP}$, there is an accepting run starting from c iff $c \in Y_{\mathcal{BP}}$.*

Intuitively, suppose there is an accepting run η from c in $\mathcal{BP}$. By definition, $c \in Y_0$. Then, each branch of η must visit an accepting control location at least once. Suppose for the i -th branch, it visits the first accepting control location at node c_i , i.e. $c_i \in F \times G^* \times 2^\delta$, by definition of Y_0 , $c_i \in Y_0$ and hence, $c_i \in Y_0 \cap F \times G^* \times 2^\delta$. Moreover, since c reaches $\bigcup_i c_i$ in η , then $c \in pre^+(\bigcup_i c_i)$, and thus, $c \in Y_1$. Since each branch of η must visit accepting control locations at least twice, then for each c_i , there must be a run starting from c_i that visits accepting control locations at least once. Inductively, we get that $c_i \in Y_1$. And therefore, $c \in Y_2$ (because $c_i \in F \times G^* \times 2^\delta$ and $c \in pre^+(\bigcup_i c_i)$). Then, since each branch of η must visit accepting control locations infinitely many times, we can

inductively apply this logic for each $j \geq 0$, obtaining that $c \in Y_j$. And thus, since $Y_{\mathcal{BP}} = \bigcap_{j \geq 0} Y_j$, we get that $c \in Y_{\mathcal{BP}}$.

D. Computing $Y_{\mathcal{BP}}$

We show that we can efficiently compute $Y_{\mathcal{BP}}$ by constructing a kind of finite automaton $\mathcal{A}$.

Definition 4. An Alternating Multi Automaton (AMA) is a tuple $A = (Q, G, T, I, F')$, where Q is a finite set of states, G is the alphabet, T is the set of transitions of the form $q \xrightarrow{\gamma} S$, where $q \in Q$, $S \subseteq Q$, and $\gamma \in G$, $I \subseteq Q$ is the set of initial states, and $F' \subseteq Q$ is the set of accepting states.

We define the reflexive-transitive closure of the $\rightarrow$ relation as follows: $q \xrightarrow{\varepsilon}^* q$, and $q \xrightarrow{\gamma\omega}^* S$ iff $q \xrightarrow{\gamma} S'$ for some $S' \subseteq Q$, $\gamma \in G$, $\omega \in G^*$, and for each $q' \in S'$, $q' \xrightarrow{\omega}^* S_{q'}$, and $S = \bigcup_{q' \in S'} S_{q'}$.

To compute $Y_{\mathcal{BP}}$, we construct an AMA $\mathcal{A} = (Q, G, T, I, F')$, such that $I \subseteq P' \times 2^\delta \times \mathbb{N}$ is the set of enumerated states of the form $(p, \theta)^i$, where $p \in P'$ is a control location in $\mathcal{BP}$, $\theta \subseteq \delta$ is a phase in $\mathcal{BP}$, $Q = I \cup \{q_f\}$, and $F' = \{q_f\}$, where q_f is a single accepting state.

We show that we can construct $\mathcal{A}$ using Algorithm 1. To force the termination of the algorithm, we define the acceleration function pr for some set of states $S \in Q$ of $\mathcal{A}$, defined as:

$$pr^{-1}(S) = \begin{cases} \{q^k \mid q^{k+1} \in S\} \cup \{q_f\} & \text{if } q_f \in S \text{ or } \exists q^1 \in S \\ \{q^k \mid q^{k+1} \in S\} & \text{else} \end{cases}$$

$$pr^k(S) = \{q^k \mid \exists j, 1 \leq j \leq k : q^j \in S\} \cup \{q_f \mid q_f \in S\}$$

The $pr^{-1}(S)$ function projects states in S of the form q^k onto states of the form q^{k-1} , such that rules of the form q^1 are projected onto q_f . The $pr^k(S)$ function projects states of the form q^j in S for any j onto states of the form q^k without changing q_f . We can show that (the proof can be found in Appendix C):

Theorem 3. Algorithm 1 terminates and computes an AMA $\mathcal{A}$, s.t. there is a path in $\mathcal{A}$ of the form $(p, \theta)^k \xrightarrow{w} \{q_f\}$ iff $(\langle p, w \rangle, \theta) \in Y_{\mathcal{BP}}$.

Construction of $\mathcal{A}$ follows the computation steps of $Y_{\mathcal{BP}} = \bigcap_{i \geq 0} Y_i$. Intuitively, $\mathcal{A}$ is a kind of a collection of AMAs A_i (that contain states of the form q^i) for $i \geq 0$, such that a configuration $(\langle p, \omega \rangle, \theta) \in Y_i$ iff $(p, \theta)^i \xrightarrow{\omega}^* \{q_f\}$ in $\mathcal{A}$. Let the set of configurations $(\langle p, \omega \rangle, \theta)$, such that $(p, \theta)^i \xrightarrow{\omega}^* \{q_f\}$ in $\mathcal{A}$ at a given point of execution be denoted as $L(A_i)$. The algorithm consists of two loops: $loop_1$ and $loop_2$. After the i -th iteration of $loop_1$, $\mathcal{A}$ contains states of the form q^i , such that there is a path $(p, \theta)^i \xrightarrow{\omega}^* \{q_f\}$ iff $(\langle p, \omega \rangle, \theta) \in Y_i$. This is ensured as follows. First, since $Y_0 = P' \times G^* \times \delta$, Line 1 creates transitions such that $Y_0 = L(A_0)$. Then, for each iteration i of $loop_1$, Line 4 adds such transitions that $L(A_i) = L(A_{i-1}) \cap F \times G^* \times 2^\delta$, which corresponds to $Y_{i-1} \cap F \times G^* \times 2^\delta$. Then, $loop_2$ computes the set of predecessors pre^* , such that after $loop_1$, $L(A_i) = pre^*(L(A_{i-1}) \cap F \times G^* \times 2^\delta)$, which corresponds to $pre^*(Y_{i-1} \cap F \times G^* \times 2^\delta)$.

Algorithm 1 Computation of $Y_{\mathcal{BP}_i}$

Input: An SM-ABPDS $\mathcal{BP}_i = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, F, \delta)$.
Output: An AMA $\mathcal{A}_i = (Q, G, T, I, \{q_f\})$.

- 1: $k := 0$, $T := \{q_f \xrightarrow{\gamma} \{q_f\} \mid \forall \gamma \in \Gamma', \forall p \in P'_i \text{ and } \forall \theta \subseteq \delta, (p, \theta)^0 := q_f\}$;
- 2: **repeat**(we call this $loop_1$)
- 3: $k := k + 1$;
- 4: For all $p \in F, \theta \subseteq \delta$, add $(p, \theta)^k \xrightarrow{\varepsilon} \{(p, \theta)^{k-1}\} \in T$;
- 5: **repeat**(we call this $loop_2$)
- 6: **for all** $\theta \subseteq \delta$, $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1 w_1, \dots, p_h w_h\} \in \Delta$ **do**
- 7: let $\sigma_j = (\rho_j, \rho'_j)$ and $\theta_j = (\theta \setminus \rho_j) \cup \rho'_j$;
- 8: let $W_j = \{R_j \mid \rho_j \subseteq \theta, (p_j, \theta_j)^k \xrightarrow{em(\theta, w_j)}^* R_j\}$;
- 9: $T := T \cup \{(p, \theta)^k \xrightarrow{\gamma} \bigcup_{R \in W} R \mid W \in \prod_{j=1}^h W_j\}$;
- 10: **end for**
- 11: **for all** $p\chi \hookrightarrow p'\gamma \in \text{restore}$, s.t. $\exists (p', \theta) \xrightarrow{\gamma} R$ **do**
- 12: $T := T \cup \{(p, \theta')^k \xrightarrow{\chi^\theta} R \mid \theta' \subseteq \delta\}$;
- 13: **end for**
- 14: **for all** $p\chi \hookrightarrow p'\gamma \in \text{discard}$, s.t. $\exists (p', \theta) \xrightarrow{\gamma} R$ **do**
- 15: $T := T \cup \{(p, \theta)^k \xrightarrow{\chi^{\theta'}} R \mid \theta' \subseteq \delta\}$;
- 16: **end for**
- 17: **until** No new transitions can be added;
- 18: For all $p \in F, \theta \subseteq \delta$, remove $(p, \theta)^k \xrightarrow{\varepsilon} \{(p, \theta)^{k-1}\}$ from T ;
- 19: For all $p \in P', \theta \subseteq \delta, \gamma \in G, R \subseteq Q$, replace $(p, \theta)^k \xrightarrow{\gamma} R$ by $(p, \theta)^k \xrightarrow{\gamma} pr^k(R)$;
- 20: **until** $k > 1$ and $\forall p \in P', \gamma \in G, \theta \subseteq \delta, R \subseteq P' \times 2^\delta \times \{k\} \cup \{q_f\}, (p, \theta)^k \xrightarrow{\gamma} R \in T \iff (p, \theta)^{k-1} \xrightarrow{\gamma} pr^{-1}(R) \in T$;

Then, Line 18 removes the initial rules added on Line 4. Since these rules were added before $loop_2$, we obtain that $L(A_i) = pre^+(L(A_{i-1}) \cap F \times G^* \times 2^\delta)$, which corresponds to $Y_i = pre^+(Y_{i-1} \cap F \times G^* \times 2^\delta)$.

Therefore, from Theorems 1, 2, and 3, we can show that:

Theorem 4. We can construct an AMA $\mathcal{A}$, such that for any configuration $(\langle p, w \rangle, \theta)$ of $\mathcal{P}$ and any BranchCARET formula ψ , $(\langle p, w \rangle, \theta) \models \psi$ iff $([p, \psi], \theta) \xrightarrow{w}^* \{q_f\}$ in $\mathcal{A}$.

E. Model Checking with Regular Valuations

Sometimes it is required to reason about stack contents in the BranchCARET atomic propositions. We can describe the stack using regular languages:

Definition 5. For an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta, \Delta^c)$, a set of configurations W is regular if: $(\langle p, \omega \rangle, \theta) \in W$ iff $\omega \in L_p$, where $L_p \subseteq \Gamma^*$ is a regular set for each $p \in P$.

A regular valuation is a function $\nu : AP \rightarrow 2^{Conf}$ that assigns to each atomic proposition a regular set of

configurations. To incorporate regular valuations into the model checking process, we can extend the construction algorithm given in Section III-B. Let $\mathcal{M}_p$ be a finite automaton that accepts L_p . The extension is straightforward and follows the lines of [23], which consists in popping the whole stack according to the transitions in $\mathcal{M}_p$ when $\mathcal{BP}$ is at the control location of the form $[p, a]$, where $a \in AP$. For a configuration $(\langle p, \gamma_1 \gamma_2 \gamma_3 \# \rangle, \theta)$ in $\mathcal{P}$ and a path in $\mathcal{M}_p$ of the form $q_1 \xrightarrow{\gamma_1} q_2 \xrightarrow{\gamma_2} q_3 \xrightarrow{\gamma_3} q_4$, would correspond to the execution of $\mathcal{BP}$ of the form $(\langle [p, a], \gamma_1 \gamma_2 \gamma_3 \# \rangle, \theta) \Rightarrow (\langle [q_1, a], \gamma_1 \gamma_2 \gamma_3 \# \rangle, \theta) \Rightarrow (\langle [q_2, a], \gamma_2 \gamma_3 \# \rangle, \theta) \Rightarrow (\langle [q_3, a], \gamma_3 \# \rangle, \theta) \Rightarrow (\langle [q_4, a], \# \rangle, \theta)$. Thus, we can show that:

Theorem 5. *We can effectively model check SM-PDS against a BranchCARET formula with regular valuations.*

F. Performance concerns

Algorithm 2 Optimized computation of $Y_{\mathcal{BP}_i}$

Input: An SM-ABPDS $\mathcal{BP}_i = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, F, \delta)$.
Output: An AMA $\mathcal{A}_i = (Q, G, T, I, \{q_f\})$.

- 1: $k := 0$, $T := \{q_f \xrightarrow{\gamma} \{q_f\} \mid \forall \gamma \in \Gamma'\}$, $\forall p \in P'_i$ and $\forall \theta \subseteq \delta$, $(p, \theta)^0 := q_f$;
- 2: **repeat**(we call this loop_1)
- 3: $k := k + 1$;
- 4: For all $p \in F$, $\theta \subseteq \delta$, add $(p, \theta)^k \xrightarrow{\varepsilon} \{(p, \theta)^{k-1}\} \in T$;
- 5: **repeat**(we call this loop_2)
- 6: **for all** $\theta \subseteq \delta$, $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_n]} \{p_1 w_1, \dots, p_h w_h\} \in \tau$, and $1 \leq j \leq h$ **do**
- 7: let $\sigma_j = (\rho_j, \rho'_j)$ and $\theta_j = (\theta \setminus \rho_j) \cup \rho'_j$;
- 8: **if** $w_j = \gamma_j \chi_j$, $\chi_j = [\gamma'_j, g_j^0, \gamma_j^0]$, and $(p_j, \theta_j)^k \xrightarrow{\gamma_j} R'_j$ **then**
- 9: let $W_j = \{S \mid ([g, \phi], \theta)^k \in R'_j, ([g, \phi], \theta)^k \xrightarrow{\gamma_j} S\} \cup \{S \mid ([p^c, QX^c \phi], \theta')^k \in R'_j, ([g_j^0, \phi], \theta)^k \xrightarrow{\gamma_j^0} S\} \cup \{S \mid ([p^c, \phi], \theta')^k \in R'_j, \phi \neq QX^c \phi', ([g_j^0, \phi], \theta)^k \xrightarrow{\gamma_j^0} S\}$;
- 10: **else**
- 11: let $W_j = \{R_j \mid \rho_j \subseteq \theta, (p_j, \theta_j)^k \xrightarrow{w_j} R_j\}$;
- 12: **end if**
- 13: $T := T \cup \{(p, \theta)^k \xrightarrow{\gamma} \bigcup_{R \in W} R \mid W \in \prod_{j=1}^h W_j\}$;
- 14: **end for**
- 15: **until** No new transitions can be added;
- 16: For all $p \in F$, $\theta \subseteq \delta$, remove $(p, \theta)^k \xrightarrow{\varepsilon} \{(p, \theta)^{k-1}\}$ from T ;
- 17: For all $p \in P'$, $\theta \subseteq \delta$, $\gamma \in G$, $R \subseteq Q$, replace $(p, \theta)^k \xrightarrow{\gamma} R$ by $(p, \theta)^k \xrightarrow{\gamma} pr^k(R)$;
- 18: **until** $k > 1$ and $\forall p \in P', \gamma \in G, \theta \subseteq \delta, R \subseteq P' \times 2^\delta \times \{k\} \cup \{q_f\}$, $(p, \theta)^k \xrightarrow{\gamma} R \in T \iff (p, \theta)^{k-1} \xrightarrow{\gamma} pr^{-1}(R) \in T$;

Algorithm 1 presented in Section III-D is generic and works with any kind of SM-ABPDS with checkpoints. However, the kind of $\mathcal{BP}$ we construct has some restrictions that allow us to optimize loop_2 . This optimization is possible because *restore* and *discard* rules are constructed in a specific manner. Let $Q \in \{E, A\}$. Consider Line 8 of the algorithm. It finds paths of the form $(p_j, \theta_j)^k \xrightarrow{em(\theta, w_j)} R_j$. If $w_j = \gamma_j \chi_j$, then the algorithm will look first for transitions of the form $(p_j, \theta_j)^k \xrightarrow{\gamma_j} R'_j$, and then, for each $q_j^l \in R'_j$, the algorithm will look for transitions of the form $q_j^l \xrightarrow{\chi_j} R_j^l$, such that $R_j = \bigcup_l R_j^l$. The transitions of the form $q_j^l \xrightarrow{\chi_j} R_j^l$ can exist in $\mathcal{A}$ only if they were added on either Line 12 or 15 (we will later explain why the two sets of transitions added by these two cases are disjoint):

- If it was added on Line 15, then $q_j^l = (p, \theta')$ for some $p \in P'$ and $\theta' \subseteq \delta$, and there is $p\chi \hookrightarrow p'\gamma \in \text{discard}$, and $(p', \theta') \xrightarrow{\gamma} R'_j$. By the construction of *discard* rules in $\mathcal{BP}$, which all have the form $[g, \phi][\gamma, p_0, \gamma_0] \hookrightarrow [g, \phi]\gamma$, p must be equal to p' , and $p = p' = [g, \phi]$ for $g \in P$, $\phi \in Cl(\psi)$, and χ has the form of $[\gamma, g_0, \gamma_0]$ (with the same γ as in right-hand side). Therefore, given that $q_j^l = ([g, \phi], \theta')$ and $\chi = [\gamma, g_0, \gamma_0]$, $q_j^l \xrightarrow{\chi} R_j^l$ exists iff $q_j^l \xrightarrow{\gamma} R'_j$ exists.
- On the other hand, if the transition $q_j^l \xrightarrow{\chi_j} R_j^l$ was added on Line 12, then $q_j^l = (p, \theta')$, there is $p\chi \hookrightarrow p'\gamma \in \text{restore}$, and $(p', \theta') \xrightarrow{\gamma} R'_j$. By the construction of *restore* rules in $\mathcal{BP}$, p must have a form of $[p^c, \phi]$, p' must be of the form $[p_0, \phi']$ for $p_0 \in P$, and either $\phi' = \phi$, or $\phi = QX^c \phi'$ ($Q \in \{E, A\}$), and χ must be of the form $[\gamma', p_0, \gamma]$. Therefore, given that $q_j^l = ([p^c, QX^c \phi'], \theta')$ and $\chi = [\gamma', p_0, \gamma]$, then $q_j^l \xrightarrow{\chi} R_j^l$ exists iff $([p_0, \phi'], \theta) \xrightarrow{\gamma} R'_j$ exists. On the other hand, given that $q_j^l = ([p^c, \phi], \theta')$, $\phi \neq QX^c \phi'$, and $\chi = [\gamma', p_0, \gamma]$, then $q_j^l \xrightarrow{\chi} R_j^l$ exists iff $([p_0, \phi], \theta) \xrightarrow{\gamma} R'_j$ exists.

Sets of transitions added by these cases are disjoint since the first one requires that $q_j^l = ([g, \phi], \theta')$ for $g \in P$, and the second one requires that $q_j^l = ([p^c, \phi], \theta')$. Therefore, we can optimize Algorithm 1 as follows:

- 1) Remove loops on Lines 14 and 11.
- 2) On line 8, if $em(\theta, w_j) = \gamma_j \chi_j^0$ and $\chi_j = [\gamma'_j, g_j^0, \gamma_j^0]$, then let $(p_j, \theta_j)^k \xrightarrow{\gamma_j} R'_j$. Then, $W_j = \{S \mid ([g, \phi], \theta)^k \in R'_j, ([g, \phi], \theta)^k \xrightarrow{\gamma_j} S\} \cup \{S \mid ([p^c, QX^c \phi], \theta')^k \in R'_j, ([g_j^0, \phi], \theta)^k \xrightarrow{\gamma_j^0} S\} \cup \{S \mid ([p^c, \phi], \theta')^k \in R'_j, \phi \neq QX^c \phi', ([g_j^0, \phi], \theta)^k \xrightarrow{\gamma_j^0} S\}$.

Therefore, by removing loops on lines 14 and 11, we significantly reduce the amount of transitions added to T . We show the optimized variant of $Y_{\mathcal{BP}}$ computation in Algorithm 2.

Proposition 5.1. *Algorithm 2 is equivalent to Algorithm 1*

under the given construction of BP .

In the hindsight, we could have evaded computing *restore* and *discard* rules from the start, jumping directly to Algorithm 2. However, this leap is highly unintuitive because then, the SM-ABPDS computed would not make any sense by itself. One could argue that even using the checkpoints is unnecessary because it is always possible to compute the caller successor during pre^* computation in $loop_2$. However, the exact implementation of a method without checkpoints would not be so apparent if we did not make such a detour.

IV. EXPERIMENTS

We have implemented our algorithm using Zig programming language and performed practical experiments.

A. Our Algorithm vs. Naive Approach

It was shown that given an SM-PDS, one can construct an equivalent standard non self-modifying PDS [24]. And an efficient algorithm for BranchCARET model checking of standard PDS was proposed in [16], [21]. Therefore, a naive approach for BranchCARET model checking of SM-PDS would be to construct an equivalent PDS, and use the existing algorithms. However, this approach is not efficient. We have implemented the naive approach and compared it with our algorithm. We generated a random SM-PDS $\mathcal{P}$ and a random BranchCARET formula ψ . Then, we ran both algorithms to model check $\mathcal{P}$ against ψ on a machine with Intel Core i3-4150 CPU @ 3.50GHz with 10 GB available memory. We recorded the execution time and the maximum memory consumed during execution. We repeated this process multiple times and presented the results in Table I. Our algorithm shows significant improvement in both speed and memory consumption (up to 30x faster and 40x less memory required). We highlighted in the table the cases showing significant improvements in memory or time. Moreover, when we consider large SM-PDS models (≈ 800 rules) or long BranchCARET formulas ($\approx 7-9$ operators), the naive approach fails either because of timeout, or because of lack of memory, while our algorithm finishes in seconds.

B. Application for Malware Detection

We have applied our algorithm for malware detection. For this, we gathered malware samples from public databases such as Malware Bazaar [1] and Virus Share [13]. We focus on binaries for x86 architecture under Windows, since this is the most common target for malware. Then, we have built a tool based on our algorithm that performs malware detection. It translates binary files into Control Flow Graphs (CFGs) using ANGR [22]. The obtained CFGs are translated into SM-PDS models using technique presented by [23], [24]. The technique models executable binary addresses as control locations and binary instructions as SM-PDS rules. For example, *call* instructions are modelled as *call* rules, *ret* instructions are modelled as *ret* rules, self-modifying instructions are modelled as self-modifying rules, and other instructions as *int* rules.

TABLE I
COMPARISON OF PERFORMANCE OF OUR APPROACH VS. PDS APPROACH ($|\Delta| + |\Delta^c|$ ARE NUMBERS OF STANDARD AND SELF-MODIFYING RULES RESPECTIVELY, $|\psi|$ IS THE SIZE OF BRANCHCARET FORMULA, OOM - OUT OF MEMORY).

			Our approach		PDS approach	
$ \Delta + \Delta^c $	$ \psi $		Time	Memory	T, s	Memory
74 + 7	9		1.7s	37.3 MB	21.9s	591.4 MB
117 + 7	3		0.3s	10.4 MB	6.0s	362.8 MB
132 + 8	5		1.0s	23.3 MB	17.8s	540.1 MB
113 + 11	3		0.6s	14.7 MB	19.4s	721.6 MB
75 + 12	3		0.3s	16.0 MB	6.1s	400.2 MB
104 + 12	9		4.4s	73.3 MB	46.7s	1.5 GB
114 + 12	5		0.4s	18.4 MB	17.1s	838.0 MB
81 + 14	5		0.6s	23.9 MB	12.4s	687.3 MB
75 + 15	7		2.1s	40.7 MB	22.2s	830.1 MB
93 + 15	5		0.4s	17.2 MB	7.1s	651.0 MB
90 + 17	9		2.4s	53.8 MB	17.5s	766.3 MB
78 + 19	5		0.7s	14.9 MB	9.7s	601.0 MB
78 + 20	5		2.3s	70.5 MB	16.6s	920.1 MB
99 + 20	5		1.1s	37.3 MB	24.4s	1.6 GB
104 + 11	9		0.8s	18.9 MB	Timeout	Timeout
108 + 13	7		9.3s	91.6 MB	OOM	OOM
105 + 18	7		0.7s	27.3 MB	OOM	OOM
68 + 20	9		9.8s	35.2 MB	Timeout	Timeout
89 + 19	7		1.9s	42.7 MB	Timeout	Timeout
118 + 20	9		11.5s	116.2 MB	OOM	OOM
86 + 19	9		2.7s	52.2 MB	Timeout	Timeout
1083 + 18	5		88.4s	4.2 GB	OOM	OOM
1224 + 16	9		112.5s	3.1 GB	OOM	OOM
677 + 19	3		13.7s	661.4 MB	OOM	OOM
714 + 12	3		11.5s	748.5 MB	43.0s	3.2 GB
1390 + 8	7		43.4s	1.6 GB	OOM	OOM
851 + 14	9		135.3s	4.2 GB	OOM	OOM
1280 + 17	5		64.4s	3.9 GB	OOM	OOM

Then, we specified several malicious behaviours using BranchCARET formulas. We executed the tool to check if the SM-PDS model of the binary satisfies the given BranchCARET formula. If it does, i.e. the model contains the malicious behaviour, the tool marked the binary as malicious. We illustrated the detected malware in Table II. First, let us present BranchCARET formulas for malicious behaviour we used. We use operators of the forms QG^b and QF^b , where $Q \in \{E, A\}$ and $b \in \{g, a, c\}$, to represent *globally* ($QG^b\phi = Q[FalseR^b\phi]$) and *eventually* ($QF^b\phi = Q[TrueU^b\phi]$) operators. Furthermore, it is important to recognize system calls in the BranchCARET formula. Sometimes, malware obfuscates calls to system functions using a chain of jump instructions, e.g. as in this code:

```

d0:  push 0
d1:  call [d]
d2:  ...
d :  jmp [GetModuleFileNameA]

```

The code obfuscates instruction `call [GetModuleFileNameA]` that calls Windows function `GetModuleFileNameA`. Instead, it calls a non-existent function with address `d`, while the address `d` contains a single jump instruction to the function `GetModuleFileNameA`.

Thus, for some function *func*, let $call(func) = call \wedge AX^gA[jmpUgfunc]$ be a shortcut formula specifying that the Windows function *func* is called. It means that the current

TABLE II
MALWARE DETECTION RESULTS.

Malware	Rules #	ψ	Time
Virus.Win32.Sp1t.c.exe	1332	ψ_s	581.12s
Trojan-Downloader.Win32.Small.axw.exe	1376	ψ_s	89.28s
P2P-Worm.Win32.Compux.a.exe	355	ψ_s	19.46s
IRC-Worm.Win32.Azaco.c.exe	425	ψ_s	23.06s
Trojan-Downloader.Win32.Small.bdw.exe	1568	ψ_s	124.15s
Virus.Win32.Mooder.l.exe	623	ψ_s	59.90s
Virus.Win32.Smog.e.exe	127	ψ_s	2.76s
Backdoor.Win32.SubSeven.22.b2.exe	2238	ψ_s	590.01s
Trojan-Downloader.Win32.Small.hj.exe	1729	ψ_t	387.94s
Trojan-Downloader.Win32.Small.xg.exe	743	ψ_t	806.43s
Trojan-Downloader.Win32.Small.cho.exe	659	ψ_t	241.60s
Trojan.Win32.KillAV.bx.exe	457	ψ_k	18.85s
Trojan-Downloader.Win32.Small.cho.exe	659	ψ_t	217.29s
Virus.Win32.Mooder.i.exe	623	ψ_s	32.55s
Trojan-Spy.Win32.Cusrev.a.exe	557	ψ_k	15.31s
Virus.Win32.Yopper.exe	92	ψ_s	1.15s
Virus.Win32.Invictus.exe	140	ψ_s	1.05s
Virus.Win32.Morgoth.2560.exe	106	ψ_s	0.85s
Virus.Win32.Mooder.a.exe	623	ψ_s	32.94s
Virus.Win32.Kanban.a.exe	140	ψ_s	1.06s
Trojan-Downloader.Win32.Botten.exe	726	ψ_t	2785.14s
Trojan.Win32.StartPage.ay.exe	3489	ψ_k	252.44s
Trojan-Downloader.Win32.Sherlol.exe	1294	ψ_t	1360.80s
Virus.Win32.Mooder.g.exe	236	ψ_s	4.08s
Trojan.Win32.LOADER.WPW.exe	298	ψ_t	13.80s
Trojan-Downloader.Win32.Small.bdc.exe	539	ψ_t	39.18s
Trojan-Downloader.Win32.Small.ef.exe	1615	ψ_t	269.74s
Trojan-Downloader.Win32.Harnig.e.exe	1921	ψ_t	866.80s
Trojan.Win32.KillAV.bb.exe	657	ψ_k	163.42s
Trojan-Dropper.Win32.Small.ir.exe	650	ψ_k	33.48s
Virus.Win32.Mooder.d.exe	236	ψ_s	6.70s
Virus.Win32.Mooder.c.exe	623	ψ_s	53.07s
Backdoor.Win32.DTR.a.exe	455	ψ_t	135.22s
Trojan-Downloader.Win32.Harnig.a.exe	2396	ψ_t	1589.98s
Trojan.Win32.Avkillah.a.exe	4986	ψ_k	441.23s
Virus.Win32.Mooder.b.exe	623	ψ_s	52.52s
Trojan-Dropper.Win32.Small.kj.exe	1090	ψ_k	3471.64s
Trojan-Downloader.Win32.Small.fn.exe	1760	ψ_t	386.02s
Trojan.Win32.KillAV.bc.exe	457	ψ_k	80.71s
Virus.Win32.Fela.8192.exe	97	ψ_s	0.72s
Trojan-Downloader.Win32.Aphex.10.d.exe	662	ψ_k	167.18s
Trojan-Downloader.Win32.Harnig.av.exe	1197	ψ_t	940.48s
Trojan.Win32.OpenPort.c.exe	373	ψ_t	101.40s

instruction is `call`, and $\mathbf{AX}^g$ refers to the called address. Then, $\mathbf{A}[jmp \mathbf{U}^g func]$ means that all paths lead to the address of `func`, and all instruction on the paths are `jmp` instructions.

1) *Trojan downloader behaviour*: Trojan downloaders are programs that are designed to deliver other malicious files to the target computer. They download the malicious files from the network and execute them immediately. To download files, programs must open a socket using `socket` function, and then receive the files using `recv` function, which takes the opened socket as a parameter. Then, the downloaded file gets executed using the `CreateProcess` function. This behaviour can be described using the following formula (we consider the disjunction $\bigvee_{d \in \Gamma}$ over all values d possibly located on the

stack):

$$\psi_t = \bigvee_{d \in \Gamma} \mathbf{EG}^g \left[\mathbf{EF}^g [call(socket) \wedge \mathbf{EX}^a eax = d \wedge \mathbf{EF}^a [call(recv) \wedge d\Gamma^* \wedge \mathbf{EF}^g call(CreateProcess)]]] \right]$$

Intuitively, this formula describes that there is a loop, such that in the future, $call(socket) \wedge \mathbf{EX}^a eax = d \wedge \mathbf{EF}^a [call(recv) \wedge d\Gamma^* \wedge \mathbf{EF}^g call(CreateProcess)]$ is satisfied. Subformula $call(socket) \wedge \mathbf{EX}^a eax = d$ means that the `socket` function is called, and at the return location of `socket` (i.e. at the abstract successor of the call), the value of the register `eax` is equal to d . Usually, the value of the `eax` register right after returning from a function contains the return value of the function. That is, the subformula means that the return value of the `socket` function (the opened socket) is equal to d . Then, the subformula $\mathbf{EF}^a [call(recv) \wedge d\Gamma^* \wedge \mathbf{EF}^g call(CreateProcess)]$ means that in the future of the same procedural context (in which `socket` was called), the subformula $call(recv) \wedge d\Gamma^* \wedge \mathbf{EF}^g call(CreateProcess)$ is satisfied. The subformula $call(recv)$ states that the `recv` function is called, and the subformula $d\Gamma^*$ means that the stack contains d on the top (we can reason about stack contents in such a way thanks to regular valuations introduced in Section III-E). Since parameters of functions are passed on the stack, these two formulas together mean that the `recv` function is called with d as parameter. Then, subformula $\mathbf{EF}^g call(CreateProcess)$ states that the `CreateProcess` function will be called in the future.

2) *Spyware behaviour*: [21] introduced the BCaRet formula describing spyware behaviour. Spyware is a type of malware that collects files matching certain conditions. To perform file searching, spyware first calls function `FindFirstFileA`, which searches files in the given folder. If the call to `FindFirstFileA` fails, then spyware obtains the reason for the failure using function `GetLastError`. Otherwise, the `FindFirstFileA` function returns a search handle d that points to the first found item. The found item may be a folder or a file. If d is a folder, the spyware calls `FindFirstFileA` again on the found folder to search for files inside of the subfolder. If d is a file, the spyware obtains a handle to the next file matching the conditions using the `FindNextFileA` function with d as parameter. This behaviour can be described as follows:

$$\psi_s = \bigvee_{d \in \Gamma} \mathbf{EF}^g [call(FindFirstFileA) \wedge \mathbf{EX}^a eax = d \wedge \mathbf{AF}^a [call(GetLastError) \vee call(FindFirstFileA) \vee (call(FindNextFileA) \wedge d\Gamma^*)]]]$$

This formula states that in the future, formula $call(FindFirstFileA) \wedge \mathbf{EX}^a eax = d \wedge \mathbf{AF}^a [call(GetLastError) \vee call(FindFirstFileA) \vee (call(FindNextFileA) \wedge d\Gamma^*)]$ is satisfied. The subformula $call(FindFirstFileA) \wedge \mathbf{EX}^a eax = d$ states that the `FindFirstFileA` function is called, and at the return location from `FindFirstFileA`, the value of the `eax` register is equal to d . Then, subformula $\mathbf{AF}^a [call(GetLastError) \vee$

$call(FindFirstFileA) \vee (call(FindNextFileA) \wedge d\Gamma^*)]$ states that in all possible futures, in the same procedure context, either the *GetLastError* function is called, the *FindFirstFileA* function is called, or formula $call(FindNextFileA) \wedge d\Gamma^*$ is satisfied. The formula $call(FindNextFileA) \wedge d\Gamma^*$ states that the *FindNextFileA* function is called with d as its parameter.

3) *Anti-antivirus behaviour*: Some malware attempts to evade antivirus programs by first shutting down the antivirus and then executing the malicious payload. To terminate the antivirus process, first, malware obtains the list of currently running processes using the *CreateToolhelp32Snapshot* function, which returns a handle d to the list. Then, it calls the *Process32First* function with d as parameter to get the first process from the list. Then, it iterates over the list by calling the *Process32Next* function with d as parameter in a loop. Then, if it finds the antivirus program, it calls the *OpenProcess* function to obtain access to the antivirus program. Then, the malware shuts down the antivirus by calling the *TerminateProcess* function. Such behaviour can be described using the following formula:

$$\begin{aligned} \psi_k = & \bigvee_{d \in \Gamma} \mathbf{EF}^g \left[call(CreateToolhelp32Snapshot) \wedge \right. \\ \mathbf{EX}^{a, e, a, x} = & d \wedge \mathbf{EF}^g \left[call(Process32First) \wedge d\Gamma^* \wedge \mathbf{AG}^g \right. \\ & \left. \mathbf{EF}^g [call(Process32Next) \wedge d\Gamma^* \wedge \mathbf{EF}^g [\right. \\ & \left. call(OpenProcess) \wedge \mathbf{AF}^a call(TerminateProcess)]] \right] \end{aligned}$$

The formula states that eventually, the subformula $call(CreateToolhelp32Snapshot) \wedge \mathbf{EX}^{a, e, a, x} = d$ will be satisfied. This subformula states that the *CreateToolhelp32Snapshot* function is called, and the return value of the function is equal to d . Then, the subformula $call(Process32First) \wedge d\Gamma^*$ states that the *Process32First* function is called with d as parameter. Then, the subformula $\mathbf{AG}^g \mathbf{EF}^g [call(Process32Next) \wedge d\Gamma^* \wedge \mathbf{EF}^g [call(OpenProcess) \wedge \mathbf{AF}^a call(TerminateProcess)]]$ states that there is a loop ($\mathbf{AG}^g \mathbf{EF}^g$), in which the function *Process32Next* is called with d as parameter, and in the future, the subformula $\mathbf{EF}^g [call(OpenProcess) \wedge \mathbf{AF}^a call(TerminateProcess)]$ is satisfied. This subformula describes that in the future, the *OpenProcess* function is called, and after that, in all possible runs the *TerminateProcess* function is called within the same procedural context.

V. CONCLUSION

In this work, we considered BranchCARET model checking of SM-PDS, which is a natural model of self-modifying binary code. We proposed a *direct* and *efficient* algorithm for model checking. We performed practical experiments, showing that our approach is faster and requires less memory than the naive approach, which consists in translating the SM-PDS into a standard PDS. Furthermore, we were able to detect

several malwares by performing model checking of SM-PDS models of binary programs against BranchCARET formulas specifying malicious behaviours. Although our approach focuses on SM-PDS, a similar approach can be applied for BranchCARET model checking of symbolic PDS because the problem presents the same challenges as SM-PDS, i.e. an implicit state that is modified along the execution, but not explicitly stated in the pushdown rules.

REFERENCES

- [1] abuse.ch: Malware Bazaar. <https://bazaar.abuse.ch/about/>, accessed: 2025-04-20
- [2] Alur, R., Etessami, K., Madhusudan, P.: A temporal logic of nested calls and returns. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems. pp. 467–481. Springer (2004)
- [3] Anckaert, B., Madou, M., De Bosschere, K.: A model for self-modifying code. In: Information Hiding: 8th International Workshop, IH 2006, Alexandria, VA, USA, July 10–12, 2006. Revised Selected Papers 8. pp. 232–248. Springer (2007)
- [4] Battista, P., Mercaldo, F., Nardone, V., Santone, A., Visaggio, C.A., et al.: Identification of android malware families with model checking. In: ICISSP. pp. 542–547 (2016)
- [5] Beaucamps, P., Gnaedig, I., Marion, J.Y.: Abstraction-based malware analysis using rewriting and model checking. In: Computer Security—ESORICS 2012: 17th European Symposium on Research in Computer Security, Pisa, Italy, September 10–12, 2012. Proceedings 17. pp. 806–823. Springer (2012)
- [6] Blackham, B., Heiser, G.: Sequoia: a framework for model checking binaries. In: 2013 IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS). pp. 97–106. IEEE (2013)
- [7] Blazy, S., Laporte, V., Pichardie, D.: Verified abstract interpretation techniques for disassembling low-level self-modifying code. Journal of Automated Reasoning **56**, 283–308 (2016)
- [8] Bonfante, G., Fernandez, J., Marion, J.Y., Rouxel, B., Sabatier, F., Thierry, A.: Codisasm: Medium scale concolic disassembly of self-modifying binaries with overlapping instructions. In: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security. pp. 745–756 (2015)
- [9] Bonfante, G., Marion, J.Y., Reynaud-Plantey, D.: A computability perspective on self-modifying programs. In: 2009 Seventh IEEE International Conference on Software Engineering and Formal Methods. pp. 231–239. IEEE (2009)
- [10] Cai, H., Shao, Z., Vaynberg, A.: Certified self-modifying code. In: Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation. pp. 66–77 (2007)
- [11] Chipounov, V., Kuznetsov, V., Candea, G.: S2e: A platform for in-vivo multi-path analysis of software systems. Acm Sigplan Notices **46**(3), 265–278 (2011)
- [12] Coogan, S.K., Debray, K.P., Townsend, G.M.: On the semantics of self-unpacking malware code. Citeseer, Tech. Rep. (2008)
- [13] Corvus Forensics: Virus Share. <https://virusshare.com/about>, accessed: 2025-04-20
- [14] David, R., Bardin, S., Ta, T.D., Mounier, L., Feist, J., Potet, M.L., Marion, J.Y.: Binsec/se: A dynamic symbolic execution toolkit for binary-level analysis. In: 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER). vol. 1, pp. 653–656. IEEE (2016)
- [15] Gorna, K., Iooss, N., Seurin, Y., Khatoun, R.: Zorya: Automated concolic execution of single-threaded go binaries. Symposium on Applied Computing (2026)
- [16] Gutsfeld, J.O., Müller-Olm, M., Nordhoff, B.: A branching time variant of caret. In: International Symposium on Model Checking Software. pp. 153–170. Springer (2018)
- [17] Kinder, J., Katzenbeisser, S., Schallhart, C., Veith, H.: Detecting malicious code by model checking. In: Detection of Intrusions and Malware, and Vulnerability Assessment: Second International Conference, DIMVA 2005, Vienna, Austria, July 7–8, 2005. Proceedings 2. pp. 174–187. Springer (2005)
- [18] King, J.C.: Symbolic execution and program testing. Communications of the ACM **19**(7), 385–394 (1976)

- [19] Korczynski, D.: Repeconstruct: reconstructing binaries with self-modifying code and import address table destruction. In: 2016 11th International Conference on Malicious and Unwanted Software (MALWARE). pp. 1–8. IEEE (2016)
- [20] Nguyen, H.V., Touili, T.: CARET model checking for malware detection. In: Proceedings of the 24th ACM SIGSOFT international SPIN symposium on model checking of software. pp. 152–161 (2017)
- [21] Nguyen, H.V., Touili, T.: Branching temporal logic of calls and returns for pushdown systems. In: International Conference on Integrated Formal Methods. pp. 326–345. Springer (2018)
- [22] Shoshitaishvili, Y., Wang, R., Salls, C., Stephens, N., Polino, M., Dutcher, A., Grosen, J., Feng, S., Hauser, C., Kruegel, C., et al.: SoK:(state of) the art of war: Offensive techniques in binary analysis. In: 2016 IEEE symposium on security and privacy (SP). pp. 138–157. IEEE (2016)
- [23] Song, F., Touili, T.: Efficient CTL model-checking for pushdown systems. Theoretical Computer Science **549**, 127–145 (2014)
- [24] Touili, T., Ye, X.: LTL model checking of self modifying code. Formal Methods in System Design **60**(2), 195–227 (2022)

APPENDIX

A. Proof of Theorem 1

Proof. Let us define a function $Assoc$, which associates a formula $\phi \in Cl(\psi)$ and the i -th step on a run π of $\mathcal{P}$ into a configuration $(\langle [p, \phi], w' \rangle, \theta)$ of $\mathcal{BP}$. $Assoc(\phi, \pi, i) = (\langle [p, \phi], w' \rangle, \theta)$, s.t. if $w = \gamma_1 \dots \gamma_n$, then $w' = \gamma'_1 \dots \gamma'_n$, where for $1 \leq i \leq n$, $\gamma'_i = \lfloor \gamma_i, p_0, \gamma_0 \rfloor$ if γ_i was pushed by a rule of form $p_0 \gamma_0 \xrightarrow{call} p' \gamma'_0 \gamma_i$, and $\gamma'_i = \gamma_i$ otherwise.

We prove the theorem by showing that for any run π in $\mathcal{P}$, $(\pi, i) \models \phi$ iff there is an accepting run in $\mathcal{BP}$ starting from $Assoc(\phi, \pi, i)$. The proof is by induction on ϕ :

• Base:

- 1) $\phi = a \in AP$. Let $\pi(i) = (\langle p, \gamma w \rangle, \theta)$. $(\pi, i) \models a$ iff $p \in \nu(a)$ iff $\exists [p, a] \gamma \xrightarrow{[\top]} \{[p, a] \gamma\} \in \Delta'$ (by item 1) iff there is an run visiting $[p, a]$ infinitely many times iff there is an accepting run starting from $Assoc(a, \pi, i)$ (the run is accepting because $[p, a] \in F$)
- 2) $\phi = \neg a \in AP$. Let $\pi(i) = (\langle p, \gamma w \rangle, \theta)$. $(\pi, i) \models \neg a$ iff $p \notin \nu(a)$ iff $\exists [p, \neg a] \gamma \xrightarrow{[\top]} \{[p, \neg a] \gamma\} \in \Delta'$ (by item 2) iff there is an run visiting $[p, \neg a]$ infinitely many times iff there is an accepting run starting from $Assoc(\neg a, \pi, i)$ (the run is accepting because $[p, \neg a] \in F$)

• Step:

- 1) $\phi = \phi_1 \vee \phi_2$. $(\pi, i) \models \phi$ iff $(\pi, i) \models \phi_1$ or $(\pi, i) \models \phi_2$ iff there is an accepting run from $Assoc(\phi_1, \pi, i)$ or $Assoc(\phi_2, \pi, i)$ (by induction hypothesis) iff there is an accepting run from $Assoc(\phi, \pi, i)$ (from item 3).
- 2) $\phi = \phi_1 \wedge \phi_2$. Similar to case 1 using item 4.
- 3) $\phi = EX^g \phi'$. Suppose $\pi(i) = (\langle p, \gamma w \rangle, \theta)$. $(\pi, i) \models \phi$ iff $\exists \pi' \in Ext(\pi), \pi'(i+1) = (\langle p', w\omega \rangle, \theta')$ and $(\pi', i+1) \models \phi'$, iff (1) $\exists p\gamma \xrightarrow{(\rho, \rho')} p'w \in \tau$, s.t. $\theta' = (\theta \setminus \rho) \cup \rho'$, and (2) there is an accepting run starting from $Assoc(\phi', \pi', i+1)$ (by induction hypothesis). (1) iff $\exists [p, \phi] \gamma \xrightarrow{[(\rho, \rho')]} p'w' \in \Delta'$ iff $Assoc(\phi, \pi, i) \implies \{Assoc(\phi', \pi', i+1)\}$ iff there is an accepting run from $Assoc(\phi, \pi, i)$ (from (2)).

4) $\phi = AX^g \phi'$. The proof is similar to case 3.

5) $\phi = EX^a \phi'$. Suppose $\pi(i) = (\langle p, \gamma w \rangle, \theta)$. $(\pi, i) \models \phi$ iff $\exists \pi' \in Ext(\pi), \pi'(next_{\pi'}^a(i)) = (\langle p', w\omega \rangle, \theta')$ and $(\pi', next_{\pi'}^a(i)) \models \phi'$, iff either:

- $next_{\pi'}^a(i) = i+1$ iff (1): $\exists p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \tau$, s.t. $\theta' = (\theta \setminus \rho) \cup \rho'$, and (2): there is an accepting run starting from $Assoc(\phi', \pi', i+1)$ (by induction hypothesis). (1) iff $\exists [p, \phi] \gamma \xrightarrow{[(\rho, \rho')]} p'w' \in \Delta'$ iff $Assoc(\phi, \pi, i) \implies \{Assoc(\phi', \pi', i+1)\}$ iff there is an accepting run from $Assoc(\phi, \pi, i)$ (from (2)); or
- $next_{\pi'}^a(i) = k > i+1$ iff (1): $\exists p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2$ and (2): there is a path $(\langle p', \gamma_1\gamma_2\omega \rangle, \theta) \implies^* (\langle p'', \gamma_2\omega \rangle, \theta')$, s.t. $\pi'(next_{\pi'}^a(i)) = (\langle p'', \gamma_2\omega \rangle, \theta')$. (1) iff $\exists [p, \phi] \gamma \xrightarrow{[(\rho, \rho')]} \{[p', \phi]^r \gamma_1\gamma_2^r\} \in \Delta'$ (from item 7) iff $(\langle [p, \phi], \gamma\omega' \rangle, \theta) \implies \{(\langle [p', \phi]^r, \gamma_1\gamma_2^r\omega' \rangle, \theta')\}$. Then, (2) iff (3): $(\langle [p', \phi]^r \gamma_1\gamma_2^r\omega' \rangle, \theta) \implies \{(\langle [p'', \phi]^r \gamma_2^r\omega' \rangle, \theta')\}$. Then, from item 11, (4): $(\langle [p'', \phi]^r \gamma_2^r\omega' \rangle, \theta') \implies \{(\langle [p'', \phi]^r \gamma_2\omega' \rangle, \theta')\}$. Thus, ((1) and (2)) iff ((3) and (4)) iff $Assoc(\phi, \pi, i) \implies^* Assoc(\phi', \pi', next_{\pi'}^a(i))$ iff there is an accepting run from $Assoc(\phi, \pi, i)$.

6) $\phi = AX^a \phi'$. The proof is similar to case 5.

7) $\phi = E[\phi_1 U^g \phi_2]$. $(\pi, i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi, i)$, s.t. $\exists k \geq i$, $(\pi', k) \models \phi_2$ and for each $i \leq j < k$, $(\pi', j) \models \phi_1$. We prove by induction on $k - i$:

- **Base** $k = i$. (1) iff $(\pi, i) \models \phi_2$ iff there is an accepting run starting from $Assoc(\phi_2, \pi, i) = (\langle [p, \phi_2], w \rangle, \theta)$ (from induction on ϕ) iff there is an accepting run from $(\langle [p, \phi], w \rangle, \theta)$ (from rule added by item 12).
- **Step** $k > i$. (1) iff (2): $(\pi, i) \models \phi_1$ and (3): $(\pi', i+1) \models \phi$. (2) iff there is an accepting run from $Assoc(\phi_1, \pi, i) = (\langle [p, \phi_1], \gamma\omega \rangle, \theta)$ (by induction on ϕ). (3) iff there is an accepting run from $Assoc(\phi, \pi', i+1) = (\langle [p', \phi], w\omega \rangle, \theta')$ (by induction on $k - i$). $\exists \pi' \in Ext(\pi, i)$ and $\pi'(i+1) = (\langle p', w'\omega \rangle, \theta)$ iff $\exists p\gamma \xrightarrow{(\rho, \rho')}_{t} pw'$ iff $\exists [p, \phi] \gamma \xrightarrow{(\rho, \rho')} \{[p, \phi_1] \gamma, [p', \phi] w\}$ (from rule added by item 14) iff there is an accepting run from $Assoc(\phi, \pi, i)$ and there is no accepting runs from $Assoc(\phi_2, \pi, i)$.

8) $\phi = A[\phi_1 U^g \phi_2]$. The proof similar to case 7.

9) $\phi = E[\phi_1 U^a \phi_2]$. $(\pi, i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi, i)$, s.t. there is a sequence $s = s_0 s_1 \dots s_k$, $s_0 = i$, for any $j \geq 0$, $s_{j+1} = next_{\pi'}^a(s_j)$, $(\pi', s_k) \models \phi_2$ and for each $0 \leq j < k$, $(\pi', s_j) \models \phi_1$. We prove by induction on k :

- **Base** $k = 0$. (1) iff $(\pi, i) \models \phi_2$ iff there is an accepting run starting from $Assoc(\phi_2, \pi, i) =$

$(\langle [p, \phi_2], w \rangle, \theta)$ (from induction on ϕ) iff there is an accepting run from $(\langle [p, \phi], w \rangle, \theta)$ (from rule added by item 12).

- **Step** $k > 0$. (1) iff (2): $(\pi, i) \models \phi_1$ and (3): $(\pi', next_{\pi}^a(i)) \models \phi$. (2) iff there is an accepting run from $Assoc(\phi_1, \pi, i) = (\langle [p, \phi_1], \gamma\omega' \rangle, \theta)$ (by induction on ϕ). (3) iff there is an accepting run from $Assoc(\phi, \pi', next_{\pi}^a(i)) = (\langle [p', \phi], w\omega' \rangle, \theta')$ (by induction on k). $\exists \pi' \in Ext(\pi, i)$ and $\pi'(next_{\pi}^a(i)) = (\langle p', w'\omega' \rangle, \theta)$ iff either:

* $\exists p\gamma \xrightarrow{(\rho, \rho')}_{int} pw'$ iff $\exists [p, \phi]\gamma \xrightarrow{(\rho, \rho')} \{[p, \phi_1]\gamma, [p', \phi]w\}$ (from rule added by item 14) iff there is an accepting run from $Assoc(\phi, \pi, i)$, or

* $\exists p\gamma \xrightarrow{(\rho, \rho')}_{call} p''\gamma_1\gamma_2$ and (2): there is a path $(\langle p'', \gamma_1\gamma_2\omega' \rangle, \theta) \Rightarrow^* (\langle p', \gamma_2\omega' \rangle, \theta')$,

s.t. $w' = \gamma_2$. (1) iff $\exists [p, \phi]\gamma \xrightarrow{[(\rho, \rho')]} \{[p'', \phi]^r\gamma_1\gamma_2^r\} \in \Delta'$ (from item 16) iff $(\langle [p, \phi], \gamma\omega' \rangle, \theta) \Rightarrow \{(\langle [p', \phi]^r, \gamma_1\gamma_2^r\omega' \rangle, \theta')\}$.

Then, (2) iff (3): $(\langle [p'', \phi]^r\gamma_1\gamma_2^r\omega' \rangle, \theta) \Rightarrow \{(\langle [p', \phi]^r\gamma_2^r\omega' \rangle, \theta')\}$. Then, from item 18, (4): $(\langle [p', \phi]^r\gamma_2^r\omega' \rangle, \theta') \Rightarrow \{(\langle [p', \phi]\gamma_2\omega' \rangle, \theta')\}$.

Thus, ((1) and (2)) iff ((3) and (4)) iff $Assoc(\phi, \pi, i) \Rightarrow^* Assoc(\phi, \pi', next_{\pi}^a(i))$ iff there is an accepting run from $Assoc(\phi, \pi, i)$.

10) $\phi = A[\phi_1 U^a \phi_2]$. The proof similar to case 9.

11) $\phi = E[\phi_1 R^g \phi_2]$. $(\pi, i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi, i)$, s.t. $\forall k \geq i$, $(\pi', k) \not\models \phi_2$ implies that $\exists j < k$, $(\pi', j) \models \phi_1$. There are two cases: either $\forall k \geq i$, $(\pi', k) \models \phi_2$, or there is $k \geq i$, s.t. $(\pi, k) \models \phi_1 \wedge \phi_2$. The latter case is proven similarly to case 7.

What is left to show is that $\forall k \geq i$, $(\pi', k) \models \phi_2$ iff $\forall k \geq i$, there is an accepting run from $Assoc(\phi, \pi', k)$.

We will prove in both directions:

- $(\Rightarrow)$ We construct the accepting run from $Assoc(\phi, \pi, i) = (\langle [p, \phi], \gamma\omega' \rangle, \theta)$. We can construct a tree with the root $Assoc(\phi, \pi, i)$, and such that every branch is accepting using rules added by item 19. There is an infinite branch b with nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$. From each of these nodes, there is an infinite branch $b_{\phi_2} = (\langle [p', \phi_2], \omega' \rangle, \theta')$. From induction on ϕ , b_{ϕ_2} is accepting. Moreover, b is also accepting because control locations of the form $[p', \phi]$ are accepting.
- $(\Leftarrow)$ The accepting run can be constructed using rules from item 19 and represented by the tree with the root $Assoc(\phi, \pi, i)$, and each node of the form $Assoc(\phi, \pi', k)$ for $k \geq i$ having children $Assoc(\phi_2, \pi', k)$ and $Assoc(\phi, \pi', k+1)$. By induction hypothesis, $(\pi', k) \models \phi_2 \forall k \geq i$.

12) $\phi = A[\phi_1 R^g \phi_2]$. The proof similar to case 11.

13) $\phi = E[\phi_1 R^a \phi_2]$. $(\pi, i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi, i)$, s.t. for a sequence $s = s_0 s_1 \dots$, where $s_0 = i$, we have that $\forall k \geq 0$, $(\pi', s_k) \not\models \phi_2$ implies that $\exists j < k$, $(\pi', s_j) \models \phi_1$. There are two cases: either $\forall k \geq 0$, $(\pi', s_k) \models \phi_2$, or there is $k \geq 0$, s.t. $(\pi, s_k) \models \phi_1 \wedge \phi_2$. The latter case is proven similarly to case 9.

What is left to show is that $\forall k \geq 0$, $(\pi', s_k) \models \phi_2$ iff $\forall k \geq 0$, there is an accepting run from $Assoc(\phi, \pi', s_k)$. There are two cases: either s is finite, or s is infinite. We will prove in both directions:

- $(\Rightarrow)$ We construct the accepting run from $Assoc(\phi, \pi, i) = (\langle [p, \phi], \gamma\omega' \rangle, \theta)$. We can construct a tree with the root $Assoc(\phi, \pi, i)$, and such that every branch is accepting using rules added by item 21 and item 8.

If s is infinite, then there is an infinite branch b with infinitely many nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$. From each of these nodes, there is an infinite branch $b_{\phi_2} = (\langle [p', \phi_2], \omega' \rangle, \theta')$. From induction on ϕ , b_{ϕ_2} is accepting. Moreover, b is also accepting because control locations of the form $[p', \phi]$ are accepting.

If s is finite, it either ends with a *ret* statement, or an infinite loop inside a called procedure. If *ret* statement ended s at s_j , then there is an infinite branch b with j nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$, the children of the last being nodes of the form $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle True, \omega' \rangle, \theta')$. If an unfinished *call* ended s at s_j , then there is an infinite branch b with j nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$, the children of the last being nodes of the form $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle [p', \phi]^r, \omega' \rangle, \theta')$. Then, the branch starting from $(\langle [p', \phi]^r, \omega' \rangle, \theta')$ is accepting because control locations of the form $[p', \phi]^r$ are accepting and repeat infinitely many times (because the call at s_j never returns).

- $(\Leftarrow)$ The accepting run can only be constructed using rules from item 21 and 8. Let $s = s_0 s_1 \dots$ be the sequence such that $s_0 = i$ and for each $j \geq 0$, $s_{j+1} = next_{\pi'}^a(s_j)$. There are three cases:

- * there is a branch b with infinitely many nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$. Let b_j be the j -th node of such form. First, we show that $b_j = Assoc(\phi, \pi', s_j)$. By definition, $b_0 = Assoc(\phi, \pi', s_0)$. Then, for each $j \geq 0$, if b_{j+1} has the form $(\langle [p', \phi], \omega' \rangle, \theta')$, then there is

$p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \tau$ and thus, $s_{j+1} = s_j + 1$. If the child of b_j has the form $(\langle [p', \phi]^r, \omega' \rangle, \theta')$,

then there is $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'w \in \tau$. Similar to the case 5, we get that b reaches b_{j+1} iff $b_j = Assoc(\phi, \pi', s_{j+1})$.

- * There is the last b_j , such that the children of b_j are of the forms $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle True, \omega' \rangle, \theta')$. Then, the rule applied at s_j must be *ret*, and therefore, s also ends at s_j .

* There is the last b_j , such that the children of b_j are of the forms $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle [p', \phi]^r, \omega' \rangle, \theta')$. Then, the rule applied at s_j must be *call*. Since there is no more nodes of the form $\langle [p', \phi], \omega' \rangle, \theta'$, then the corresponding return was not met, and therefore, s also ends at s_j .

Each node b_j has another child of the form $(\langle [p', \phi_2], \omega' \rangle, \theta')$. Thus, from the induction on ϕ , we get that for each s_j , $(\pi', s_j) \models \phi_2$.

14) $\phi = A[\phi_1 R^a \phi_2]$. The proof similar to case 13.

15) $\phi = EX^c \phi'$. We consider two cases:

- ($next_\pi^c(i)$ is defined) Suppose $next_\pi^c(i) = k$, $\pi(i) = (\langle p_i, w \rangle, \theta_i)$, and $\pi(k) = (\langle p_k, \gamma_k \omega \rangle, \theta_k)$. Let the call at the k -th step would be of the form $p_k \gamma_k \xrightarrow{\sigma_{call}} p_{k+1} \gamma' \gamma''$. Hence, if $Assoc(\phi', \pi, k) = (\langle [p_k, \phi'], \gamma_k \omega' \rangle, \theta_k)$, then $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi], w' \omega' \rangle, \theta_i)$, s.t. $w = u \gamma''$ and $w' = \gamma_i u [\gamma'', p_k, \gamma_k]_{\theta_k}$. Item 23 adds a rule of the form $[p_i, \phi] \gamma_i \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$ and item 26 adds rules of the form $[p^c, \phi] \gamma^j \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$ for $\gamma^j \in u$. Only these rules are allowed to be applied at each control location. Hence, $(\langle [p_i, \phi], w' \omega' \rangle, \theta_i) \Rightarrow^* \{(\langle [p^c, \phi], [\gamma'', p_k, \gamma_k]_{\theta_k} \omega' \rangle, \theta_i)\}$. Then, only the rule from *restore* can be applied and it allows $(\langle [p^c, \phi], [\gamma'', p_k, \gamma_k]_{\theta_k} \omega' \rangle, \theta_i) \Rightarrow \{(\langle [p_k, \phi'], \gamma_k \omega' \rangle, \theta_k)\}$. By induction on ϕ , $(\pi, k) \models \phi'$ iff there is an accepting run from $(\langle [p_k, \phi'], \gamma_k \omega' \rangle, \theta_k) = Assoc(\phi', \pi, k)$ and hence, $(\pi, i) \models \phi$ iff there is an accepting run from $\langle [p_i, \phi], w' \omega' \rangle, \theta_i = Assoc(\phi, \pi, i)$.
- ($next_\pi^c(i)$ is undefined) Suppose $next_\pi^c(i) = \perp$. $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi], \gamma_i w \# \rangle, \theta_i)$, where $w \in \Gamma^*$. Item 23 adds a rule of the form $[p_i, \phi] \gamma_i \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$ and item 26 adds rules of the form $[p^c, \phi] \gamma^j \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$ for $\gamma^j \in w$. Only these rules can be applied at each control location. Thus, $(\langle [p_i, \phi], \gamma_i w \# \rangle, \theta_i) \Rightarrow^* \{(\langle [p^c, \phi], \# \rangle, \theta_i)\}$.

Only rule of the form $[p^c, \phi] \# \xrightarrow{[\top]} \{[p^c, \phi] \# \}$ can be applied, and since $[p^c, \phi] \notin F$, there is no accepting runs from $Assoc(\phi, \pi, i)$.

16) $\phi = E[\phi_1 U^c \phi_2]$. We prove in both directions:

- ($\Rightarrow$) Let $(\pi, i) \models \phi$, i.e. let $s = s_0 s_1 \dots s_k$ be the sequence, s.t. $s_0 = i$ and for $j \geq 0$, $s_{j+1} = next_\pi^c(s_j)$, and $(\pi, s_k) \models \phi_2$ and for $0 \leq j < k$, $(\pi, s_j) \models \phi_1$. By induction on ϕ , there are accepting runs from $Assoc(\phi_2, \pi, s_k)$ and each $Assoc(\phi_1, \pi, s_j)$. We proof by induction on k :

* **Base** $k = 0$, then $(\pi, i) \models \phi_2$, and then, there is an accepting run from $Assoc(\phi_2, \pi, i) =$

$Assoc(\phi_2, \pi, s_k)$. The rule added by item 12 creates allows the accepting run starting from $Assoc(\phi, \pi, i) \Rightarrow \{Assoc(\phi_2, \pi, i)\}$.

* **Step** $k > 0$. Then, $(\pi, i) \models \phi_1$ and $(\pi, next_\pi^c(i)) \models \phi$. By induction on ϕ , there is an accepting from $Assoc(\phi_1, \pi, i)$, and by induction on k , there is an accepting run from $Assoc(\phi, \pi, next_\pi^c(i))$. Let $\pi(i) = (\langle p_i, \gamma_i u \gamma'' \omega \rangle, \theta_i)$, s.t. γ'' was pushed to the stack at the call locations using a rule of the form $p_{s_1} \gamma_{s_1} \xrightarrow{\sigma} p' \gamma' \gamma'' \in \tau$. Thus, $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi] \gamma_i u [\gamma'', p_{s_1}, \gamma_{s_1}]_{\omega'}^{\theta_{s_1}}, \theta_i \rangle)$, where θ_{s_1} is the phase at the call location. Rule added by item 24 creates transition $Assoc(\phi, \pi, i) \Rightarrow \{Assoc(\phi_1, \pi, i), (\langle [p^c, \phi] u [\gamma'', p_{s_1}, \gamma_{s_1}]_{\omega'} \rangle, \theta_i)\}$. Then, rules added by item 26 allow $(\langle [p^c, \phi] u [\gamma'', p_{s_1}, \gamma_{s_1}]_{\omega'} \rangle, \theta_i) \Rightarrow^* \{(\langle [p^c, \phi] [\gamma'', p_{s_1}, \gamma_{s_1}]_{\omega'} \rangle, \theta_i)\}$. Then using the *restore* rules: $(\langle [p^c, \phi] [\gamma'', p_{s_1}, \gamma_{s_1}]_{\omega'} \rangle, \theta_i) \Rightarrow \{(\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})\}$. And thus, $Assoc(\phi, \pi, i) \Rightarrow \{Assoc(\phi_1, \pi, i), Assoc(\phi, \pi, next_\pi^c(i))\}$. And thus, there is an accepting run from $Assoc(\phi, \pi, i)$.

– ($\Leftarrow$) Let $Assoc(\phi, \pi, i) = (\langle [p_i, \phi] \omega \rangle, \theta_i)$. Let k be the number of checkpoints $\chi^\theta \in G$ in ω . We prove by induction on k .

* **Base** $k = 0$. Then, only transition $Assoc(\phi, \pi, i) \Rightarrow \{Assoc(\phi_2, \pi, i)\}$ is possible. By induction on ϕ , $(\pi, i) \models \phi_2$ and thus, $(\pi, i) \models \phi$.

* **Step** $k > 0$. Then, either base case applies, or the accepting run starts with $Assoc(\phi, \pi, i) \Rightarrow \{Assoc(\phi_1, \pi, i), (\langle [p^c, \phi] u [\gamma'', p_{s_1}, \gamma_{s_1}]_{\omega'}^{\theta_{s_1}} \rangle, \theta_i)\} \Rightarrow^* \{Assoc(\phi_1, \pi, i), (\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})\}$, where $\omega = \gamma u [\gamma'', p_{s_1}, \gamma_{s_1}]_{\omega'}^{\theta_{s_1}}$. Since $(\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1}) = Assoc(\phi, \pi, next_\pi^c(i))$ uses less checkpoints, by induction on k , $(\pi, next_\pi^c(i)) \models \phi$. Also, by induction on ϕ , $(\pi, i) \models \phi_1$. Thus, $(\pi, i) \models \phi$.

17) $\phi = E[\phi_1 R^c \phi_2]$. Let $s = s_0 s_1 \dots s_k$, s.t. $s_0 = i$, for $j \geq 0$, $s_{j+1} = next_\pi^c(s_j)$. We prove in both directions:

– ($\Rightarrow$) Let $(\pi, i) \models \phi$. There are two cases: either there is j , s.t. $(\pi, s_j) \models \phi_1 \wedge \phi_2$, and for each $0 \leq l < j$, $(\pi, s_l) \models \phi_1$. In this case, the proof is similar to case 16. Or, $\forall j, 0 \leq j \leq k$, $(\pi, s_j) \models \phi_2$. In this case, we prove by induction on k :

* **Base** $k = 0$. Then, $next_\pi^c(i) = \perp$ and if $\pi(i) = (\langle p, \gamma w \rangle, \theta)$, then w does not contain any return locations pushed by *call* rules. Hence, $Assoc(\phi, \pi, i) =$

$(\langle [p, \phi], \gamma w \# \rangle, \theta)$. There is a rule created by item 25, which allows $\text{Assoc}(\phi, \pi, i) \Rightarrow \{\text{Assoc}(\phi_2, \pi, i), (\langle [p^c, \phi] w \# \rangle, \theta)\}$. And rules added by item 26 allow $(\langle [p^c, \phi] w \# \rangle, \theta) \Rightarrow^* \{(\langle [p^c, \phi] \# \rangle, \theta)\}$. By induction on ϕ , there is an accepting run from $\text{Assoc}(\phi_2, \pi, i)$. And there is an accepting run from $(\langle [p^c, \phi] \# \rangle, \theta)$ created by the rule added by item 26 because $[p^c, \phi] \in F^c \subseteq F$.

* **Step** $k > 0$. By induction on ϕ , there is an accepting run from $\text{Assoc}(\phi_2, \pi, i)$. Let $\pi(i) = (\langle p_i, \gamma_i u \gamma'' \omega \rangle, \theta_i)$, s.t. γ'' was pushed to the stack at the call locations using a rule of the form $p_{s_1} \gamma_{s_1} \xrightarrow{\sigma} p' \gamma' \gamma'' \in \tau$. Thus, $\text{Assoc}(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi] \gamma_i u [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle^{\theta_{s_1}}, \theta_i)$, where θ_{s_1} is the phase at the call location. Rule added by item 25 creates transition $\text{Assoc}(\phi, \pi, i) \Rightarrow \{\text{Assoc}(\phi_2, \pi, i), (\langle [p^c, \phi] u [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle, \theta_i)\}$. Then, rules added by item 26 allow $(\langle [p^c, \phi] u [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle, \theta_i) \Rightarrow^* \{(\langle [p^c, \phi] [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle, \theta_i)\}$. Then using the *restore* rules: $(\langle [p^c, \phi] [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle, \theta_i) \Rightarrow \{(\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})\}$. By induction on k , there is an accepting run from $\text{Assoc}(\phi, \pi, \text{next}_\pi^c(i)) = (\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})$, and since $\text{Assoc}(\phi, \pi, i) \Rightarrow \{\text{Assoc}(\phi_2, \pi, i), \text{Assoc}(\phi, \pi, \text{next}_\pi^c(i))\}$, then there is an accepting run from $\text{Assoc}(\phi, \pi, i)$.

- ($\Leftarrow$) There are two cases: either $\text{Assoc}(\phi, \pi, i)$ reaches some configuration $\{\text{Assoc}(\phi_1, \pi, s_j), \text{Assoc}(\phi_2, \pi, s_j)\}$. In this case, the proof is similar to case 16. Otherwise, the run reaches configurations with nodes $\{\text{Assoc}(\phi_1, \pi, s_k), (\langle [p_{s_k}, \phi] \# \rangle, \theta_{s_k})\}$. In this case, the accepting run has the form of a tree with one branch containing k nodes of the form $(\langle [p_{s_j}, \phi], \omega' \rangle, \theta_{s_j})$, and the k -th such node has children $\text{Assoc}(\phi_2, \pi, s_k)$ and $(\langle [p^c, \phi] u \# \rangle, \theta_{s_k})$. Moreover, for each s_j , there is a child of the form $\text{Assoc}(\phi_2, \pi, s_j)$ (created by the rule added by item 25). By induction on ϕ , $(\pi, s_j) \models \phi_2$ for each $0 \leq j \leq k$, and therefore, $(\pi, i) \models \phi$. $\square$

B. Proof of Theorem 2

Proof. First, we prove the following lemma:

Lemma 6. $\mathcal{BP}$ has a run η starting from c , such that each path visits accepting control locations at least k times iff $c \in Y_k$.

Proof. We prove in both directions:

- ($\Rightarrow$) We prove by induction on k . Base case is trivial since $Y_0 = \text{Conf}_{\mathcal{BP}}$.

Let $c \Rightarrow^* \{(\langle p_1, w_1 \rangle, \theta_1), \dots, (\langle p_n, w_n \rangle, \theta_n)\}$, s.t. for $1 \leq i \leq n$, $(\langle p_i, w_i \rangle, \theta_i)$ is the first visited accepting node ($p \in F$). Then, for $1 \leq i \leq n$, $\mathcal{BP}$ has a run η_i starting from $(\langle p_i, w_i \rangle, \theta_i)$ and visiting accepting control locations at least $k-1$ times. By induction hypothesis, $(\langle p_i, w_i \rangle, \theta_i) \in Y_{k-1}$. Since $Y_{k-1} \cap (F \times G^* \times 2^\delta) \supseteq \{(\langle p_1, w_1 \rangle, \theta_1), \dots, (\langle p_n, w_n \rangle, \theta_n)\}$, then $c \in \text{pre}^*(Y_{k-1} \cap (F \times G^* \times 2^\delta)) = Y_k$.

- ($\Leftarrow$) We prove by induction on k . Base case is trivial since $Y_0 = \text{Conf}_{\mathcal{BP}}$.

Since $Y_k = \text{pre}^*(Y_{k-1} \cap (F \times G^* \times 2^\delta))$, then $c \Rightarrow^* \{(\langle p_1, w_1 \rangle, \theta_1), \dots, (\langle p_n, w_n \rangle, \theta_n)\}$, where $\{(\langle p_1, w_1 \rangle, \theta_1), \dots, (\langle p_n, w_n \rangle, \theta_n)\} \subseteq Y_{k-1} \cap (F \times G^* \times 2^\delta)$, i.e. for each $1 \leq i \leq n$, $p_i \in F$ and $(\langle p_i, w_i \rangle, \theta_i) \in Y_{k-1}$. By induction hypothesis, there is a run from $(\langle p_i, w_i \rangle, \theta_i)$ visiting accepting states at least $k-1$ times. And since $p_i \in F$, then there is a run η from c that visits accepting states at least k times. $\square$

Now we proceed with the proof of Theorem 2. We prove in both directions:

- ($\Rightarrow$) We show by proving that if the configuration c is not in $Y_{\mathcal{BP}}$, then there is no accepting runs from c in $\mathcal{BP}$. Suppose $c \notin Y_{\mathcal{BP}}$. Thus, there exists Y_k , s.t. $c \notin Y_k$. From Lemma 6, there is no run starting from c and visiting accepting control locations at least k times. Thus, there is no accepting runs from c .
- ($\Leftarrow$) Suppose $c \in Y_{\mathcal{BP}}$. It is sufficient to construct an accepting run starting from c . Since the sequence $Y_0 Y_1 \dots$ is monotone and $Y_{\mathcal{BP}} = \bigcap_{j \geq 0} Y_j$, we obtain that $Y_{\mathcal{BP}} = \text{pre}^+(Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta)$. Since $c \in \text{pre}^+(Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta)$, then there is a set of configurations $\{c_1, \dots, c_n\} \subseteq Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta$, s.t. $c \Rightarrow^* \{c_1, \dots, c_n\}$. Therefore, for each $1 \leq i \leq n$, $c_i \in Y_{\mathcal{BP}}$.

In other words, for $c \in Y_{\mathcal{BP}}$, we can construct a finite subrun η in $\mathcal{BP}$ with root c and leaves $\{c_1, \dots, c_n\}$, s.t. for each $1 \leq i \leq n$, $c_i \in Y_{\mathcal{BP}}$. Then, we can extend each c_i in η with a new subrun η_i that starts at c_i and each leaf of η_i is also in $Y_{\mathcal{BP}}$. We can repeat this process infinitely many times, thus obtaining the accepting run η . $\square$

C. Proof of Theorem 3

Proof. The proof is mostly identical to the proof of Theorem 2 in [23]. However, we use a different ABPDS model and hence, we need to prove their Lemma 3 and Lemma 4 for SM-ABPDS.

Lemma 7 corresponds to Lemma 3 in [23]:

Lemma 7. In Algorithm 1, for every $i \geq 1$, after line 18, we have that $\mathcal{L}(A_i) = \text{pre}^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$.

Proof. We prove in both directions.

$(L(A_i) \subseteq \text{pre}^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta)))$: Suppose $(\langle p, w \rangle, \theta) \in A_i$. There are no transitions of the form $(p, \theta) \xrightarrow{\varepsilon} \{q_f\}$ after line 18 therefore, $|w| \geq 1$.

We can show that during *loop*₂, if $(\langle p, w \rangle, \theta) \in A_i$, then $(\langle p, w \rangle, \theta) \in \text{pre}^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. We prove by induction on w .

Base $w = \varepsilon$. Then, we have a transition added at line 4, which means $p \in F$ and $(p, \theta)^i \xrightarrow{\varepsilon} \{(p, \theta)^{i-1}\}$. Therefore, respectively, $(\langle p, w \rangle, \theta) \in F \times G^* \times 2^\delta$ and $(\langle p, w \rangle, \theta) \in L(A_{i-1})$. And therefore, the base case holds.

Step There are two cases:

- $w = \gamma u$ for $\gamma \in \Gamma, u \in G^*$. Thus, there exist $S \subseteq Q$, s.t. $t = (p, \theta)^i \xrightarrow{\gamma} S \in T$, $S \xrightarrow{u}^* \{q_f\}$. t could be added to T at the only one place on line 9, which means $\exists p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_m]} \{p_1 w_1, \dots, p_m w_m\}$, $\sigma_j = (\rho_j, \rho'_j)$, $S = \{q \in S_j \mid \rho_j \subseteq \theta \text{ and } (p_j, \theta_j)^l \xrightarrow{w_j}^* S_j\}$, where $\theta_j = (\theta \setminus \rho_j) \cup \rho'_j$. By applying the induction hypothesis, for every j , $1 \leq j \leq m$, if $\rho_j \subseteq \theta$, then $(\langle p_j, w_j u \rangle, \theta_j) \in \text{pre}^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. Thus, we get that $(\langle p, w \rangle, \theta) \in \text{pre}^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$.
- $w = \chi^{\theta'} u$ for $\chi \in X, u \in G^*, \theta' \subseteq \delta$. Thus, there exist $S \subseteq Q$, s.t. $t = (p, \theta)^i \xrightarrow{\chi^{\theta'}} S \in T$, $S \xrightarrow{u}^* \{q_f\}$. t could be added to T at two places:
 - On line 12, which means $\exists p\chi \hookrightarrow p'\gamma \in \text{restore}$, $(p', \theta')^l \xrightarrow{\gamma}^* S$. By applying the previous item, we have that $(\langle p', \gamma u \rangle, \theta') \in \text{pre}^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. Thus, we get that $(\langle p, w \rangle, \theta) \in \text{pre}^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$.
 - On line 15, which means $\exists p\chi \hookrightarrow p'\gamma \in \text{discard}$, $(p', \theta')^l \xrightarrow{\gamma}^* S$. By applying the previous item, we have that $(\langle p', \gamma u \rangle, \theta) \in \text{pre}^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. Thus, we get that $(\langle p, w \rangle, \theta) \in \text{pre}^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$.

Since the algorithm removes ε transitions on line 18, we know that $|w| \geq 1$. We can apply the same induction but with at least one step to obtain $(\langle p, w \rangle, \theta) \in \text{pre}^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$.

$$(\text{pre}^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))) \subseteq L(A_i):$$

Since $\text{pre}^+(W) = \text{pre}^*(\text{pre}(W))$ for any $W \subseteq \text{Conf}$, we obtain that $\text{pre}^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$ is the limit of the infinite sequence $\{C_j\}_{j \geq 0}$, such that $C_0 = \text{pre}(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$ and $C_{j+1} = C_j \cup \text{pre}(C_j)$ for every j , $j \geq 0$. Because $C_j \subseteq C_{j+1}$ for every $j \geq 0$, it is sufficient to show that for every tuple $(\langle p, w \rangle, \theta) \in C_j$, we have a path $(p, \theta) \xrightarrow{w}^* \{q_f\}$ in A_i that does not use transition rules of the form $q^i \xrightarrow{\varepsilon} \{q^{i-1}\}$ for any $q^i, q^{i-1} \in Q$. We prove by induction.

Base $j = 0$. By applying the saturation procedure at *loop*₂, A_i accepts C_0 if only the states of the form $(p, \theta)^i$ are treated as initial states, which are the outgoing states of the added transitions during *loop*₂. Since these transitions are of the form

$(p, \theta)^i \xrightarrow{\gamma} S$ for $\gamma \in G$, $Q \subseteq P \times 2^\delta \times \{i-1\} \cup \{q_f\}$, we have that for every $(\langle p, w \rangle, \theta) \in C_0$, $(\langle p, w \rangle, \theta) \in L(A_i)$.

Step $j \geq 1$. Since $C_j = C_{j-1} \cup \text{pre}(C_{j-1})$, we get that either $(\langle p, w \rangle, \theta) \in C_{j-1}$ or $(\langle p, w \rangle, \theta) \in \text{pre}(C_{j-1})$. In the first case, the step holds from the induction hypothesis. Otherwise, if $(\langle p, w \rangle, \theta) \in \text{pre}(C_{j-1})$, there are two cases depending on what rule was applied:

- $\exists p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_m]} \{p_1 w_m, \dots, p_m w_m\} \in \Delta'$, such that $w = \gamma u$ and for every j , $1 \leq k \leq m$, let $\sigma_k = (\rho_k, \rho'_k)$, if $\rho_k \subseteq \theta$, then $(\langle p_k, \text{em}(\theta, w_k)u \rangle, \theta_k) \in C_{j-1}$, where $\theta_k = (\theta \setminus \rho_k) \cup \rho'_k$. We apply the induction hypothesis to get $(p_k, \theta_k)^i \xrightarrow{\text{em}(\theta, w_k)u}^* \{q_f\}$. Let us deconstruct the path as follows: $(p_k, \theta_k)^i \xrightarrow{\text{em}(\theta, w_k)}^* Q_j \xrightarrow{u}^* \{q_f\}$. Therefore, the saturation procedure will add the transition $(p, \theta) \xrightarrow{\gamma} S$, where $S = \{q \in Q_k \mid \rho_k \subseteq \theta\}$. Thus, we get that $(\langle p, w \rangle, \theta) \in L(A_i)$.
- $\exists p\chi \hookrightarrow p'\gamma \in \text{restore}$, $w = \chi^{\theta'} u$, $(\langle p', \gamma u \rangle, \theta') \in C_{j-1}$ for some $\theta' \subseteq \delta$. We apply the induction hypothesis to get $(p', \theta')^i \xrightarrow{\gamma u}^* \{q_f\}$. Let us deconstruct the path as follows: $(p', \theta')^i \xrightarrow{\gamma}^* S \xrightarrow{u}^* \{q_f\}$. Therefore, the saturation procedure will add the transition $(p, \theta) \xrightarrow{\chi^{\theta'}} S$. Thus, we get that $(\langle p, w \rangle, \theta) \in L(A_i)$.
- $\exists p\chi \hookrightarrow p'\gamma \in \text{discard}$, $w = \chi^{\theta'} u$, for some $\theta' \subseteq \delta$, and $(\langle p', \gamma u \rangle, \theta) \in C_{j-1}$. We apply the induction hypothesis to get $(p', \theta')^i \xrightarrow{\gamma u}^* \{q_f\}$. Let us deconstruct the path as follows: $(p', \theta')^i \xrightarrow{\gamma}^* S \xrightarrow{u}^* \{q_f\}$. Therefore, the saturation procedure will add the transition $(p, \theta) \xrightarrow{\chi^{\theta'}} S$. Thus, we get that $(\langle p, w \rangle, \theta) \in L(A_i)$.

Therefore, $\text{pre}^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta)) \subseteq L(A_i)$. $\square$

Lemma 8 corresponds to Lemma 4 in [23]:

Lemma 8. Consider an **Algorithm C**, which is **Algorithm 1** without line 20 (without termination condition of *loop*₁). In **Algorithm C**, for every $i \geq 0$:

- for every accepting run η in $\mathcal{BP}$ starting from $(\langle p, w \rangle, \theta)$, there is a path $(p, \theta)^i \xrightarrow{w}^* \{q_f\}$ in A_i and for every decomposition $(p, \theta)^i \xrightarrow{u} Q \xrightarrow{v}^* \{q_f\}$ of the path, if $Q \neq \{q_f\}$, then for all $(p', \theta')^i \in Q \setminus \{q_f\}$, η visits $(\langle p', v \rangle, \theta')$;
- After substitution at line 19, $Y_{\mathcal{BP}} \subseteq L(A_i)$.

Proof. We proceed by induction on i .

Base $i = 0$. The statement (a) holds from the fact that for any $p \in P'$, $\theta \subseteq \delta$, $w \in G^*$, $(p, \theta)^0 \xrightarrow{w}^* \{q_f\}$ during $i = 0$ and for any decomposition $(p, \theta)^i \xrightarrow{u} Q \xrightarrow{v}^* \{q_f\}$, $Q = \{q_f\}$. The statement (b) holds from the fact that $\mathcal{L}(A_0) = P' \times G^* \times 2^\delta$.

Step $i \geq 1$. For the statement (a). Let $H(\eta)$ be the maximum number of steps over branches of η to reach some accepting control location. We proceed by induction on $H(\eta)$.

- **Base** $H(\eta) = 0$. Since the root of η is $(\langle p, w \rangle, \theta)$, then we have that $(\langle p, w \rangle, \theta) \in F \times G^* \times 2^\delta$. The line 1 adds

transition $(p, \theta)^i \xrightarrow{\varepsilon} \{(p, \theta)^{i-1}\}$ and the result follows from the induction hypothesis on i .

- **Step** $H(\eta) \geq 1$. Let the root node of η , $(\langle p, w \rangle, \theta)$, have m child runs $\eta_1, \dots, \eta_m$, s.t. for every j , $1 \leq j \leq m$, η_j starts at $(\langle p_j, w_j \rangle, \theta_j)$. There are three cases depending on which rule was applied:

- $\exists p\gamma \xrightarrow{[\sigma_{j_1}, \dots, \sigma_{j_h}]} \{p_{j_1}w_{j_1}, \dots, p_{j_h}w_{j_h}\} \triangleright \{D_{j_1}, \dots, D_{j_h}\}$, s.t. $\{1, \dots, m\} \subseteq \{j_1, \dots, j_h\}$, $w = \gamma w'$, and for every j , $1 \leq j \leq m$, $\theta_j = (\theta \setminus \rho_j) \cup \rho'_j$, $\sigma_j = (\rho_j, \rho'_j)$, and $\rho_j \subseteq \theta$. We apply the induction on $H(\eta_j)$ for every j , $1 \leq j \leq m$, to obtain that there is a path $(p_j, \theta_j)^i \xrightarrow{em(\theta, w_j)w'}^* \{q_f\}$ in A_i and for every decomposition $(p_j, \theta_j)^i \xrightarrow{u}^* Q_j \xrightarrow{v}^* \{q_f\}$, if $Q_j \neq \{q_f\}$, then for every $(p', \theta')^i$ or $(p', \theta')^{i-1} \in Q_j \setminus \{q_f\}$, η_j reaches a node labelled $(\langle p', v \rangle, \theta')$.

Let $(p_j, \theta_j)^i \xrightarrow{w_j}^* Q'_j \xrightarrow{w'}^* \{q_f\}$. After application of the saturation at line 9, we get:

$$(p, \theta)^i \xrightarrow{\gamma}^* \bigcup_{j=1}^m Q'_j \xrightarrow{w'}^* \{q_f\}$$

For every decomposition $(p, \theta)^i \xrightarrow{u}^* Q \xrightarrow{v}^* \{q_f\}$, $Q \subseteq \bigcup_{j=1}^m Q_j$, which means that for every state of the form $(p', \theta')^i$ or $(p', \theta')^{i-1} \in Q \setminus \{q_f\}$, η reaches a node labelled $(\langle p', v \rangle, \theta')$.

- $\exists p\chi \hookrightarrow p_1\gamma \in \text{restore}$, s.t. $m = 1$, $w = \chi^{\theta'}\omega$ for some θ' , and $(\langle p_1, \gamma\omega \rangle, \theta')$ is the only child node. We apply the induction hypothesis on $H(\eta_1)$ to obtain that there is a path $(p_1, \theta')^i \xrightarrow{\gamma\omega}^* \{q_f\}$ in A_i , and for every decomposition of the form $(p_1, \theta')^i \xrightarrow{u}^* Q \xrightarrow{v}^* \{q_f\}$, if $Q \neq \{q_f\}$, then for every $(p', \theta'')^i$ or $(p', \theta'')^{i-1} \in Q \setminus \{q_f\}$, η_1 reaches a node labelled $(\langle p', v \rangle, \theta'')$.

Let $(p_1, \theta')^i \xrightarrow{\gamma}^* Q' \xrightarrow{\omega}^* \{q_f\}$. After application of the saturation at line 9, we get:

$$(p, \theta)^i \xrightarrow{\chi^{\theta'}}^* Q' \xrightarrow{\omega}^* \{q_f\}$$

Therefore, since $(p, \theta)^i \xrightarrow{w}^* \{q_f\}$, we have that for every decomposition $(p, \theta)^i \xrightarrow{u}^* Q \xrightarrow{v}^* \{q_f\}$, $Q \subseteq \bigcup_{j=1}^m Q_j$, which means that for every state of the form $(p', \theta'')^i$ or $(p', \theta'')^{i-1} \in Q \setminus \{q_f\}$, η reaches a node labelled $(\langle p', v \rangle, \theta'')$.

- $\exists p\chi \hookrightarrow p_1\gamma \in \text{discard}$, s.t. $m = 1$, $w = \chi^{\theta'}\omega$ for some θ' , and $(\langle p_1, \gamma\omega \rangle, \theta)$ is the only child node. We apply the induction hypothesis on $H(\eta_1)$ to obtain that there is a path $(p_1, \theta)^i \xrightarrow{\gamma\omega}^* \{q_f\}$ in A_i , and for every decomposition of the form $(p_1, \theta)^i \xrightarrow{u}^* Q \xrightarrow{v}^* \{q_f\}$, if $Q \neq \{q_f\}$, then for every $(p', \theta'')^i$ or $(p', \theta'')^{i-1} \in Q \setminus \{q_f\}$, η_1 reaches a node labelled $(\langle p', v \rangle, \theta'')$.

Let $(p_1, \theta)^i \xrightarrow{\gamma}^* Q' \xrightarrow{\omega}^* \{q_f\}$. After application of the saturation at line 9, we get:

$$(p, \theta)^i \xrightarrow{\chi^{\theta'}}^* Q' \xrightarrow{\omega}^* \{q_f\}$$

Therefore, since $(p, \theta)^i \xrightarrow{w}^* \{q_f\}$, we have that for every decomposition $(p, \theta)^i \xrightarrow{u}^* Q \xrightarrow{v}^* \{q_f\}$, $Q \subseteq \bigcup_{j=1}^m Q_j$, which means that for every state of the form $(p', \theta'')^i$ or $(p', \theta'')^{i-1} \in Q \setminus \{q_f\}$, η reaches a node labelled $(\langle p', v \rangle, \theta'')$.

Thus, the statement (a) holds.

For the statement (b). First, we need to show that $Y_{\mathcal{BP}} \subseteq L(A_1)$. From Lemma 7, we have that $L(A_1) = pre^+(L(A_0) \cap F \times G^* \times 2^\delta)$, i.e. $L(A_1) = pre^+(F \times G^* \times 2^\delta)$ before line 19. Since $Y_1 = pre^+(F \times G^* \times 2^\delta)$, it is sufficient to prove that for every $k \geq 1$, $(\langle p, w \rangle, \theta) \in Y_{\mathcal{BP}}$, there is a path $(p, \theta)^k \xrightarrow{w}^* \{q_f\}$ in A_k after line 19. This is proven in what follows.

Suppose $k \geq 2$. We apply the induction hypothesis on k to obtain that $Y_{\mathcal{BP}} \subseteq L(A_{k-1})$.

Since $Y_{\mathcal{BP}} = pre^+(Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta)$, we get that $Y_{\mathcal{BP}} \subseteq pre^+(L(A_{k-1}) \cap F \times G^* \times 2^\delta)$. From Lemma 7, $L(A_k) = pre^+(L(A_{k-1}) \cap F \times G^* \times 2^\delta)$ after line 18. Therefore, we prove that for every $k \geq 1$, if $(\langle p, w \rangle, \theta) \in Y_{\mathcal{BP}}$, then there is a path $(p, \theta)^k \xrightarrow{w}^* \{q_f\}$ in A_k after line 19. Let m be the number of transitions substituted at line 19. For l , $0 \leq l \leq m$, let A_k^l be the automaton obtained after substituting l transitions. We proceed by induction on l .

- **Base** $l = 0$. We directly get that $Y_{\mathcal{BP}} \subseteq A_k^0$.
- **Step** $l \geq 1$. We apply the induction hypothesis to obtain that $Y_{\mathcal{BP}} \subseteq L(A_k^{l-1})$. If $L(A_k^{l-1}) \subseteq L(A_k^l)$, the item holds from the fact that $Y_{\mathcal{BP}} \subseteq A_k^{l-1}$. Otherwise, suppose there is $(\langle p, w \rangle, \theta) \in L(A_k^{l-1}) \setminus L(A_k^l)$, such that $|w| = \min\{|w'| \mid (\langle p', w' \rangle, \theta') \in L(A_k^{l-1}) \setminus L(A_k^l)\}$. We prove by contradiction that $(\langle p, w \rangle, \theta) \notin Y_{\mathcal{BP}}$. Since $(p, \theta)^k \xrightarrow{w}^* \{q_f\}$ in A_k^{l-1} , let $u \in G^+$, $v \in G^*$, $p' \in P'$, $\theta' \subseteq \delta$, $Q' \subseteq Q$, such that $w = uv$, $(p, \theta) \xrightarrow{u}^* Q' \cup \{(p', \theta')^{k-1}\} \xrightarrow{v}^* \{q_f\}$ in A_k^{k-1} , and there is no such path $(p', \theta')^k \xrightarrow{v}^* \{q_f\}$ in A_k^l (otherwise, $(\langle p, w \rangle, \theta) \in L(A_k^l)$). By the statement (a), the accepting run must reach the local configuration $(\langle p', v \rangle, \theta')$ during the accepting run. It is sufficient to show that $(\langle p', v \rangle, \theta') \notin Y_{\mathcal{BP}}$. Since $(\langle p', v \rangle, \theta') \in L(A_k^{m-1})$, then we have that $(\langle p', v \rangle, \theta') \in L(A_k^{m-1}) \setminus L(A_k^m)$. Since $loop_2$ can only add new transitions, we get that $|v| > |w|$, which contradicts the definition of $|w|$.

Thus, the statement (b) holds. □

□